

Freestanding Proposal

Document number: P0829R3

Date: 2018-10-06

Reply-to: Ben Craig <ben dot craig at gmail dot com>

Audience: SG14, Library Evolution Working Group

Change history

R2 -> R3

Changed feature test macro to better align with P1105.
Split feature test macros into freestanding and hosted macros.
Updating to July 26, 2018 snapshot of the standard.
Added `identity`.
Added `<contract>` and `<concepts>`.
Added new `shift_left` and `shift_right` algorithms.
Added insert iterators.
Added `mbstate_t` and `wint_t` for `char_traits`.

R1 -> R2

Removed `atoi` because of reliance on `isspace`.
Added `visit` and `bad_variant_access`.
Added `bitset`, minus the string members and throwing members.
Added `<version>` and moved the feature macro there.
Added `unique_ptr`. Leaving `make_unique` and `default_delete` out.
Added `<span>`.
Added days, weeks, months, and years time durations.
Added `<csetjmp>`.
Updated to N4741.

R0 -> R1

R1 adds an abstract.
R1 no longer removes any existing freestanding libraries.
R1 no longer gives implementations freedom to add `static_asserts` to allocation functions in freestanding mode.
`unique_ptr` is no longer included in freestanding, with the hopes that `unique_resource` and `scope_exit` will fill the void
R1 adds naming alternatives for freestanding.
R1 includes substantially less of `<random>`, and white-lists inclusions rather than black-lists omissions.
R1 adds some allocator machinery, though it does not add the default allocator.
R1 adds `<compare>`, as it seems to have been accidentally omitted from the freestanding headers list.
R1 adds large portions of `<chrono>`, `<optional>`, and `<variant>`.
R1 adds a feature test macro.

Abstract

Add everything to the freestanding implementation that can be implemented without OS calls and space overhead.

Introduction

The current definition of the freestanding implementation is not very useful. Here is the current high level definition from [intro.compliance]:

- 7 Two kinds of implementations are defined: a *hosted implementation* and a *freestanding implementation*. For a hosted implementation, this document defines the set of available libraries. A freestanding implementation is one in which execution may take place without the benefit of an operating system, and has an implementation-defined set of libraries that includes certain language-support libraries ([compliance]).

The main people served by the current freestanding definition are people writing their own hosted C++ standard library to sit atop the compiler author's freestanding implementation (i.e. the STLport use case). The freestanding portions contain all the functions and types known to the compiler that can't easily be authored in a cross-compiler manner.

The current set of freestanding libraries provides too little to kernel and embedded programmers. Why should a systems programmer need to rewrite `std::sort()` or `std::memcpy()`?

I propose we provide the (nearly) maximal subset of the library that does not require an OS or space overhead. In order to continue supporting the "layered" C++ standard library users, we will continue to provide the (nearly) minimal subset of the library needed to support all the language features, even if these features have space overhead. Language features requiring space overhead or OS support will remain intact.

Motivation

Systems programmers want to sort things. They want to use move semantics. They may even want to bundle the arguments of a variadic template function into a `std::tuple`. These are all reasonable things to do on a system with no operating system and kilobytes of storage. The C++ standard even has reasonable specifications for these operations in the context of a tiny, OS-less system. However, systems programmers must currently rely on either hand-rolled code or implementer extensions in order to get these facilities.

Systems programmers don't have a guide as to what C++ library facilities will work without trying them. The standard says `atomic_load` will work; `memcpy` will probably work; but will `stable_sort`? Standardizing the subset of implementable C++ that is usable in a freestanding environment would provide clarity here, and help to educate systems programmers.

Current State

There were some presentations at recent CppCons where others made a more full featured C++ work in the Linux and Windows kernels [Quinn2016] [Baker2017]. In both of these cases, C++ was being used in a sandboxed / pseudo-virtualized way. C++ code (with exceptions and RTTI) was being run in the kernel context, but very few calls were made between the C++ code and the operating system kernel. The C++ code was in a guest operating system. This proposal should make it reasonable to have C++ code interact closely with the internals of a host operating system, perhaps in the context of a driver.

The Microsoft Windows kernel and Apple Mac OSX kernel both currently support limited, non-compliant subsets of C++ for driver writers. The Linux kernel does not support C++ officially, though with a fair amount of work on the part of the driver developer, C++ can be made to work. Drivers written in C++ are highly unlikely to be accepted in the upstream Linux source repositories.

IncludeOS [Bratterud2017] is an OS primarily intended for running in VMs, though some bare metal support has been tested. One might expect such a project to use a freestanding implementation as a base, but instead, it starts with a hosted implementation of C++ and drops support for the impractical parts (threads and filestreams in particular).

Out of `libstdc++`, `libc++`, and Microsoft's Visual Studio STL, only `libstdc++` has any relevant mention of "freestanding" or "hosted". In practice, users take a hosted implementation of C++ and use it as-is in situations where it was never intended. This means that all the headers tend to be available, but only a few of the headers actually work. Many of the headers that work aren't marked freestanding. Some headers have parts that could work, except they are mixed with other parts that won't work. For example, `iterator_traits` in `<iterator>` is fine, but the implementation details of the stream iterators cause build errors with the `/kernel` flag in Microsoft Visual Studio 2017.

Scope

The current scope of this proposal is limited to the freestanding standard library available to systems and embedded programming.

This paper is currently concerned with the divisions of headers and library functions as they were in C++17. ["Standard Library Modules" \(P0581\)](#) discusses how the library will be split up in a post-modules world. This paper may influence the direction of P0581, but this paper won't make any modules recommendations.

I could see the scope increasing to the availability of the standard library on GPUs.

Impact on the standard

The standard will no longer list all of the facilities available to the freestanding implementation, as is currently done in [compliance]. Instead, [compliance] will list all the headers that are required to be present, and the header and class synopses will tag which parts of the headers and classes will be available. This is a large number of small, easy to understand edits, along with the addition of a sub-clause that discusses how the tagging works.

There is precedent for this kind of tagging in other specification documents. The ECMAScript Language Specification has optional support for ECMA-402 (internationalization). The impact of ECMA-402 is called out explicitly in several places. POSIX tags functions as supported in base, XSI, or in one of many option groups.

There were some conversations in the 2017 Albuquerque meeting around adding another class of conforming implementation. I believe that such an action would be a mistake. Maintaining two classifications is difficult enough as is, and freestanding is often neglected. Adding another classification would magnify these problems. I also feel that the freestanding classification should be removed if no action is taken to make it more useful.

Naming alternatives

There was some desire to come up with a new name for "freestanding" in the 2017 Albuquerque meeting. This new name could better express the intended audience of such an implementation. My current recommendation will be to keep the name "freestanding", but I will suggest some alternatives just the same.

- backless
- barebones
- basic
- embedded
- foundational
- freestanding
- fundamental
- independent
- kernel
- minimal
- OS-free
- OS-less
- reduced
- resource-constrained
- skeleton
- standalone
- stripped
- system-free
- unsupported

Impact on implementations

C++ standard library headers will likely need to add preprocessor feature toggles to portions of headers that would emit warnings or errors in freestanding mode. The precision and timeliness (compile time vs. link time) of errors remains a quality-of-implementation detail.

A minimal freestanding C11 standard library will not be sufficient to provide the C portions of the C++ standard library. `std::char_traits` and many of the function specializations in `<algorithm>` are implemented in terms of non-freestanding C functions. In practice, most C libraries are not minimal freestanding C11 libraries. The optimized versions of the `<cstring>` and `<wchar>` functions will typically be the same for both hosted and freestanding environments.

My expectation is that no new freestanding library will be authored as a result of this paper. Instead hosted libraries will be stripped down through some feature toggle mechanism to become freestanding.

The Microsoft Visual Studio standard library implementation has taken recent steps in version 15.8 (released Aug 14, 2018) to try to better enable uses of the standard library in the kernel. From the [Features and Fixes blog post](#)

The header structure of the STL was changed to allow use of a subset of the library in conditions where the user can't link with `msvcrt.dll`, such as driver development. [...] The following headers are now considered "core" and don't inject our runtime dependencies (though we do still assume some form of CRT headers are present):

- `<cstdint>`
- `<cstdlib>`
- `<initializer_list>`
- `<ratio>`
- `<type_traits>`

I had the following email discussion with Jonathan Wakely, the maintainer of `libstdc++`, the gcc implementation of the C++ standard library.

From: Ben Craig

If *someone* (possibly me) added the maze of `#ifdefs` to a branch of `libstdc++` to support p0829, would the `libstdc++` maintainers be willing to accept and maintain such a thing (assuming it is in the working draft, has tests, high quality, etc)?

From: Jonathan Wakely

Yes, definitely. If it's in the WP (and even better, useful to constrained environments) we will support it.

GCC is heavily used on bare-metal systems (especially ARM) and if those users can leverage the C++ standard library then we provide more value to them.

I had similar email discussion with Marshall Clow, the maintainer of `libc++`, the clang implementation of the C++ standard library.

From: Ben Craig

[...]

If *someone* (possibly me) added the maze of `#ifdefs` to a branch of `libc++` to support p0829, would the `libc++` maintainers be willing to accept and maintain such a thing (assuming it is in the working draft, has tests, high quality, etc)? In the next draft of p0829, may I publicly state that `libc++` would be willing to add that support?

[...]

From: Marshall Clow

I haven't been ignoring you, but rather talking to a bunch of people who use `libc++`>

In general, they are in favor of supporting a freestanding mode for `libc++`. Some of them, however, have serious concerns about p0829.

So, yes, you can put my name on the list, but I suspect you'll get comments about the specifics.

Design decisions

Even more so than for a hosted implementation, systems and embedded programmers do not want to pay for what they don't use. As a consequence, I am not adding features that require global storage, even if that storage is immutable.

Note that the following concerns are not revolving around execution time performance. These are generally concerns about space overhead and correctness.

This proposal doesn't remove problematic features from the language, but it does make it so that the bulk of the freestanding standard library doesn't require those features. Users that disable the problematic features (as is existing practice) will still have portable portions of the standard library at their disposal.

Note that we cannot just take the list of `constexpr` or conditionally `noexcept` functions and make those functions the freestanding subset. We also can't do the reverse, and make everything freestanding `constexpr` or conditionally `noexcept`. `memcpy` cannot currently be made `constexpr` because it must convert from `cv void*` to `unsigned char[]`. Several floating point functions could be made `constexpr`, but would not be permitted in freestanding. The "Lakos Rule"[Meredith11] prohibits most standard library functions from being conditionally `noexcept`, unless they have a wide contract. Regardless, if a function or class is `constexpr` or `noexcept`, and it doesn't involve floating point, then that function or class is a strong candidate to be put into freestanding mode.

In the future, it may make sense to allow all `constexpr` functions into freestanding, so long as they are used in a `constexpr` context and not invoked at runtime.

Exceptions

Exceptions either require external jump tables or extra bookkeeping instructions. This consumes program storage space.

In the Itanium ABI, throwing an exception requires a heap allocation. In the Microsoft ABI, re-throwing an exception will consume surprisingly large amounts of stack space (2,100 bytes for a re-throw in 32-bit environments, 9,700 bytes in a 64-bit environment). Program storage space, heap space, and stack space are typically scarce resources in embedded development.

In environments with threads, exception handling requires the use of thread-local storage.

RTTI

RTTI requires extra data in vtables and extra classes that are difficult to optimize away, consuming program storage space.

Thread-local storage

Thread-local storage requires extra code in the operating system for support. In addition, if one thread uses thread-local storage, that cost is imposed on other threads.

The heap

The heap is a big set of global state. In addition, heap exhaustion is typically expressed via exception. Some embedded systems don't have a heap. In kernel environments, there is typically a heap, but there isn't a reasonable choice of *which* heap to use as the default. In the Windows kernel, the two best candidates for a default heap are the paged pool (plentiful available memory, but unsafe to use in many contexts), and the non-paged pool (safe to use, but limited capacity). The C++ implementation in the Windows kernel forces users to implement their own global operator `new` to make this decision.

Floating point

Many embedded systems don't have floating point hardware. Software emulated floating point can drag in large runtimes that are difficult to optimize away.

Most operating systems speed up system calls by not saving and restoring floating point state. That means that kernel uses of floating point operations require extra care to avoid corrupting user state.

Functions requiring global or thread-local storage

These functions have been omitted or removed from the freestanding library. Examples are the locale aware functions and the C random number functions.

Parallel algorithms

For the `<algorithms>`, `<numeric>`, and `<memory>` headers, we would only be able to support sequential execution of parallel algorithms. Since this adds little value, the execution policy overloads will be omitted.

Partial class inclusion

Some classes are only partially freestanding. In the Albuquerque 2017 meeting, there was no consensus on allowing or prohibiting this (SF F N A SA 0 9 3 5 1).

In this proposal, I partially include three classes: `std::array`, `std::string_view`, and `std::optional`. If we were designing these classes from scratch with a freestanding environment in mind, they would have the exact same layout and the exact same interface, with the exception of the excluded methods. I think it is highly unlikely that the committee would standardize a `std::freestanding_array` that was the exact same as `std::array`, just without the `at()` method.

I would like to call out `std::variant` as well. While I do include the entire class, I do exclude the `get()` function. `get()` is logically part of the interface. The same general argument I used for `std::array`, `std::string_view`, and `std::optional` holds for `std::variant`.

Technical Specifications

Complete headers newly required for freestanding implementations

- `<setjmp>`
- `<utility>`
- `<tuple>`
- `<ratio>`
- `<compare>`
- `<span>`
- `<contract>`
- `<concept>`

Partial headers newly required for freestanding implementations

Portions of `<cstdlib>`

- `size_t`
- `div_t`
- `ldiv_t`
- `lldiv_t`
- `NULL`
- `EXIT_FAILURE`
- `EXIT_SUCCESS`
- `_Exit`
- `bsearch`
- `qsort`
- `abs(int)`
- `abs(long int)`
- `abs(long long int)`
- `labs`
- `llabs`
- `div`
- `ldiv`
- `lldiv`

All the error `#defines` in `<cerrno>`, but not `errno`.

The `errc` enum from `<system_error>`.

All of `<optional>`, except for `bad_optional_access` and `optional::value`. The `optional` value can be accessed through the unchecked observers, `operator*` and `operator->`.

All of `<variant>`, except for `get`. The `variant` value can be accessed through `get_if`.

All of `<bitset>` except for `operator<<`; `operator>>`; `bitset::to_string`; the `bitset` string constructors; and the indexed versions of `bitset::set`, `bitset::reset`, and `bitset::flip`.

Portions of `<memory>`.

- `pointer_traits`
- `to_address`
- `align`
- `allocator_arg_t`
- `allocator_arg`
- `uses_allocator`
- `allocator_traits`
- [\[specialized.algorithms\]](#), except for the `ExecutionPolicy` overloads
- `unique_ptr`
Note that `default_delete` is not included, and wording has been added to allow the default deleter of `unique_ptr` to be implementation defined on freestanding implementations.
- `uses_allocator_v`

Most of `<functional>`. **Omit** the following.

- [\[func.wrap\]](#), polymorphic function wrappers (i.e. `std::function` and friends).
- [\[func.search.bm\]](#), `boyer_moore_searcher`
- [\[func.search.bmh\]](#), `boyer_moore_horspool_searcher`

`<chrono>` durations and duration math. **Omit** clocks, civil time, and streaming functions.

Portions of `<charconv>`.

- `to_chars_result`
- `from_chars_result`
- `to_chars(integral)`
- `from_chars(integral)`

The `char_traits` class from `<string>`.

Most of `<string_view>`. These functions will be **omitted**:

- `operator<<`
- `basic_string_view::at`
- `basic_string_view::copy`
- `basic_string_view::substr`
- `basic_string_view::compare(size_type pos1, size_type n1, basic_string_view s);`
- `basic_string_view::compare(size_type pos1, size_type n1, basic_string_view s, size_type pos2, size_type n2);`
- `basic_string_view::compare(size_type pos1, size_type n1, const charT* s);`
- `basic_string_view::compare(size_type pos1, size_type n1, const charT* s, size_type n2);`

Portions of `<cstring>`.

- `memcpy`
- `memmove`
- `strcpy`
- `strncpy`
- `strcat`
- `strncat`
- `memcmp`
- `strcmp`
- `strncmp`
- `memchr`
- `strchr`
- `strcspn`
- `strpbrk`
- `strrchr`
- `strspn`
- `strstr`
- `memset`
- `strlen`

Portions of `<wchar>`.

- `wscpy`
- `wcsncpy`
- `wmemcpy`
- `wmemmove`
- `wscat`
- `wcsncat`
- `wscmp`
- `wcsncmp`
- `wmemcmp`
- `wcschr`

- wscspn
- wcsprbrk
- wcsrchr
- wcssp
- wcsstr
- wcstok
- wmemchr
- wcslen
- wmemset

All of `<array>` except for `array::at`.

All of `<iterator>` except for the stream iterators.

Most of `<algorithm>` and `<numeric>`. The `ExecutionPolicy` overloads will not be included. The following functions will be **omitted** due to the usage of temporary buffers:

- `stable_sort`
- `stable_partition`
- `inplace_merge`

Portions of `<random>`. The following portions will be included:

- `linear_congruential_engine`
- `mersenne_twister_engine`
- `subtract_with_carry_engine`
- `discard_block_engine`
- `independent_bits_engine`
- `shuffle_order_engine`
- `[rand.predef]`
- `uniform_int_distribution`

A small portion of `<cmath>` will be present.

- `abs(int)`
- `abs(long int)`
- `abs(long long int)`

A portion of `<cinttypes>` will be present.

- `imaxabs`
- `imaxdiv`
- `abs(intmax_t)`
- `div(intmax_t, intmax_t)`

Notable omissions

`errno` is not included as it is global state. In addition, `errno` is best implemented as a thread-local variable.

`error_code`, `error_condition`, and `error_category` all have `string` in the interface.

Many string functions (`strtol` and family) rely on `errno`.

`strtok` and `rand` aren't required to use thread-local storage, but good implementations do. I don't want to encourage bad implementations.

`assert` is not included as it requires a `stderr` stream.

`<cctype>` and `<cwctype>` rely heavily on global locale data.

`default_delete` and `make_unique` are not included, although `unique_ptr` is included. `default_delete` and `make_unique` have direct dependencies on heap management functionality.

`seed_seq` uses the heap. Users can create their own classes satisfying the seed sequence requirements if they wish to use the `Sseq` constructors on the engine templates.

Potential removals

Here are some things that I am currently requiring, but could be convinced to remove.

- `<wchar>`

The `<wchar>` functions are implementable for freestanding environments, but infrequently used for systems programming. They do not exist in freestanding C11 implementations.

- Insert iterators

`back_insert_iterator`, `front_insert_iterator`, and `insert_iterator` don't directly allocate memory, but their typical usage with standard containers does allocate memory. This paper doesn't add any standard containers that can be used with the insert iterators. It is also unclear how a user

would signal an insertion failure. Regardless, the iterators could be used with a user's (or future standard) `fixed_capacity_vector`.

- `visit`

`visit` only throws when the `variant` is valueless. If exceptions are disabled, the `variant` will always have a value. Since the standard doesn't acknowledge exceptions being disabled, we include the (hopefully) dead `bad_variant_access` class.

- `unique_ptr`

`unique_ptr` is generally used for heap management, but is occasionally used as a makeshift RAI object for other resources. In order to break the compile time dependency on `default_delete`, I add some implementation defined behavior for the freestanding implementation. This may be too invasive for some committee members' tastes. In the Jacksonville 2018 wg21 meeting, there was a strong interest in providing `unique_ptr`.

- Partial classes

`array`, `bitset`, `optional`, `string_view`, and `variant` all have functions that throw. These functions aren't critical to the use of the class. The exceptions that `array`, `bitset`, and `string_view` throw are doubly bad, as those exceptions require `std::string`. These headers and classes can be omitted if the committee strongly objects to marking a class partially freestanding.

Potential additions

Here are some things that I am not currently requiring, but could be convinced to add.

- `complex<integer>`

I currently omit the `complex` class entirely, but in theory, the integer version of `complex` are fine. I am unsure how many of the associated functions are implementable without floating point access though. I do not believe that `complex` for integer types is a widely used class.

- Floating point support

Perhaps we don't worry about library portability in all cases. Just because kernel modes can't easily use floating point doesn't mean that we should deny floating point to the embedded space. Do note that most of `<cmath>` has a dependency on `errno`.

- `errno` and string functions like `strtol`

While `errno` is global data, it isn't much global data. Thread safety is a concern for those platforms that have threading, but don't have thread-local storage.

Feature Test Macros

A freestanding implementation that provides support for this paper shall define the following feature test macro.

Name	Value	Header
<code>__cpp_freestanding_library</code>	<code>201803</code>	<code><version></code>

The `__cpp_freestanding_library` macro is useful for detecting the absence of throwing facilities. A user could conditionally use a hand-rolled implementation in freestanding mode or the standard implementation in hosted mode.

The library feature macros have been partitioned into those that must be present in both freestanding and hosted mode, and those that only need to be present in hosted mode. If the implementation provides more than the minimal freestanding subset, then the implementation shall also provide the corresponding feature test macros.

The pre-existing feature macros were not provided on freestanding / hosted boundaries. As a result, a choice has to be made about feature test macros that contain both freestanding and non-freestanding code (e.g. `__cpp_lib_chrono`, `__cpp_lib_boyer_moore_searcher`, and others). This revision includes feature test macros in freestanding if any of the feature is included in freestanding. Users that need to detect the non-freestanding parts will also need to test against `__cpp_freestanding_library`.

Wording

Wording is based off of an intermediate version of Working Draft, Standard for Programming Language C++, from July 26, 2018.

Add a new subclause [freestanding.membership], under [conventions] and after [objects.within.classes]:

???? **Freestanding membership** [freestanding.membership]

1 The freestanding implementation has several declarations and macro definitions that shall meet the same requirements as for a hosted implementation unless otherwise specified. In the associated header synopsis for such declarations and macro definitions, the items shall be followed with a comment that ends with *freestanding*, as in:

#define E2BIG see below //freestanding.

- 2 The freestanding implementation has several headers that shall meet the same requirements as for a hosted implementation unless otherwise specified. The synopsis for these headers shall start with a comment that ends with *freestanding*, as in:

//freestanding
namespace std {

Individual declarations and macro definitions in such a header shall not be followed with a comment that ends with *freestanding*.

- 3 Some classes and class templates are required to be partially implemented in a freestanding implementation. Such entities are permitted to be fully implemented in a freestanding implementation. In the associated header synopsis for such declarations, the declarations shall be followed with a comment that ends with *freestanding, partial*, as in:

template<class T>
class optional; //freestanding, partial

Some member functions in partially implemented entities are not required to be present in a freestanding implementation. In the associated class or class template synopsis for such member functions, the declarations shall be followed by a comment that ends with *freestanding, omit*, as in:

constexpr const T& value() const&; //freestanding, omit

All declarations in the partially implemented class or class template that are not followed by a *freestanding, omit* comment shall meet the same requirements as for a hosted implementation unless otherwise specified.

- 4 Deduction guides for class templates that are implemented (partially or fully) in a freestanding implementation shall be implemented in a freestanding implementation. The containing namespace for each non-namespace declaration that is implemented (partially or fully) in a freestanding implementation shall be implemented in a freestanding implementation.

Rework Table 21 referenced from [compliance]:

Table 21 - C++ headers for freestanding implementations

	Subclause	Header(s)
{support.types}	Types	<cstdint>
{support.limits}	Implementation properties	<float> <limits> <limits> <version>
{cstdint}	Integer types	<stdint>
{support.start.term}	Start and termination	<stdlib>
{support.dynamic}	Dynamic memory management	<new>
{support.rtti}	Type identification	<typeinfo>
{support.exception}	Exception handling	<exception>
{support.initlist}	Initializer lists	<initializer_list>
{support.runtime}	Other runtime support	<cstdlib>
{meta}	Type traits	<type_traits>
{bit}	Bit manipulation	<bit>
{atomics}	Atomics	<atomic>

Table 21 - C++ headers for freestanding implementations

<algorithm>	<cmath>	<exception>	
<array>	<compare>	<functional>	<string>
<atomic>	<concepts>	<initializer_list>	<string_view>
<bit>	<contract>	<iterator>	<system_error>
<bitset>	<csetjmp>	<limits>	<tuple>
<cerrno>	<cstdlib>	<memory>	<type_traits>
<cfloat>	<cstdint>	<new>	<typeinfo>
<charconv>	<cstdint>	<numeric>	<utility>
<chrono>	<cstdlib>	<optional>	<variant>
<cinttypes>	<cstring>	<random>	<version>
<climits>	<wchar>	<ratio>	

Change in [compliance] paragraph 3:

~~The supplied version of the header <stdlib> shall declare at least the functions abort, atexit, at_quick_exit, exit, and quick_exit [support.start.term]. The other headers listed in this table shall meet the same requirements as for a hosted implementation. The~~

supplied versions of the header <version> shall meet the requirements specified in [support.limits.general]. The other headers listed in this table shall meet the requirements for a freestanding implementation, as specified in the respective header synopsis.

Instructions to the editor:

Please add a `// freestanding` comment to the beginning of the following synopses. These headers are entirely freestanding.

- [cstddef.syn]
- [limits.syn]
- [climits.syn]
- [cfloat.syn]
- [cstdint.syn]
- [climits.syn]
- [cstdint.syn]
- [new.syn]
- [typeinfo.syn]
- [exception.syn]
- [initializer_list.syn]
- [compare.syn]
- [cstdarg.syn]
- [csetjmp.syn]
- [concepts.syn]
- [utility.syn]
- [tuple.syn]
- [meta.type.synop]
- [ratio.syn]
- [span.syn]
- [bit.syn]
- [atomics.syn]

Change in [cstdlib.syn]:

???	Header <cstdlib> synopsis	[cstdliblib.syn]
<pre>namespace std { using size_t = see below; // freestanding using div_t = see below; // freestanding using ldiv_t = see below; // freestanding using lldiv_t = see below; // freestanding } #define NULL see below // freestanding #define EXIT_FAILURE see below // freestanding #define EXIT_SUCCESS see below // freestanding #define RAND_MAX see below #define MB_CUR_MAX see below namespace std { // Exposition-only function type aliases extern "C" using c-atexit-handler = void(); // exposition only extern "C++" using atexit-handler = void(); // exposition only extern "C" using c-compare-pred = int(const void*, const void*); // exposition only extern "C++" using compare-pred = int(const void*, const void*); // exposition only // [support.start.term], start and termination [[noreturn]] void abort() noexcept; // freestanding int atexit(c-atexit-handler* func) noexcept; // freestanding int atexit(atexit-handler* func) noexcept; // freestanding int at_quick_exit(c-atexit-handler* func) noexcept; // freestanding int at_quick_exit(atexit-handler* func) noexcept; // freestanding [[noreturn]] void exit(int status); // freestanding [[noreturn]] void _Exit(int status) noexcept; // freestanding [[noreturn]] void quick_exit(int status) noexcept; // freestanding char* getenv(const char* name); int system(const char* string); // [c.malloc], C library memory allocation void* aligned_alloc(size_t alignment, size_t size); void* calloc(size_t nmemb, size_t size); void free(void* ptr);</pre>		

```

void* malloc(size_t size);
void* realloc(void* ptr, size_t size);

double atof(const char* nptr);
int atoi(const char* nptr);
long int atol(const char* nptr);
long long int atoll(const char* nptr);
double strtod(const char* nptr, char** endptr);
float strtof(const char* nptr, char** endptr);
long double strtold(const char* nptr, char** endptr);
long int strtol(const char* nptr, char** endptr, int base);
long long int strtoll(const char* nptr, char** endptr, int base);
unsigned long int strtoul(const char* nptr, char** endptr, int base);
unsigned long long int strtoull(const char* nptr, char** endptr, int base);

// [c.mb.wcs], multibyte / wide string and character conversion functions
int mblen(const char* s, size_t n);
int mbtowc(wchar_t* pwc, const char* s, size_t n);
int wctomb(char* s, wchar_t wchar);
size_t mbstowcs(wchar_t* pwcs, const char* s, size_t n);
size_t wcstombs(char* s, const wchar_t* pwcs, size_t n);

// [alg.c.library], C standard library algorithms
void* bsearch(const void* key, const void* base, size_t nmemb, size_t size,
              c-compare-pred* compar); // freestanding
void* bsearch(const void* key, const void* base, size_t nmemb, size_t size,
              compare-pred* compar); // freestanding
void qsort(void* base, size_t nmemb, size_t size, c-compare-pred* compar); // freestanding
void qsort(void* base, size_t nmemb, size_t size, compare-pred* compar); // freestanding

// [c.math.rand], low-quality random number generation
int rand();
void srand(unsigned int seed);

// [c.math.abs], absolute values
int abs(int j); // freestanding
long int abs(long int j); // freestanding
long long int abs(long long int j); // freestanding
float abs(float j);
double abs(double j);
long double abs(long double j);

long int labs(long int j); // freestanding
long long int llabs(long long int j); // freestanding

div_t div(int numer, int denom); // freestanding
ldiv_t div(long int numer, long int denom); // see [library.c], freestanding
lldiv_t div(long long int numer, long long int denom); // see [library.c], freestanding
ldiv_t ldiv(long int numer, long int denom); // freestanding
lldiv_t lldiv(long long int numer, long long int denom); // freestanding
}

```

Replace paragraph 3 in [support.limits.general]:

- 3 The macros in Table 35 are defined after inclusion of the header `<version>` or one of the corresponding headers specified in the table. **[Note: Future versions of this International Standard might replace the values of these macros with greater values. —end note]**
- 3 **On freestanding and hosted implementations, the macros in Table ??35?? are defined after inclusion of the header `<version>` or one of the corresponding headers specified in the table. On hosted implementations, the macros in Table ??36?? are defined after inclusion of the header `<version>` or one of the corresponding headers specified in the table. The presence of the macros in Table ??36?? is implementation defined on freestanding implementations.** **[Note: Future versions of this International Standard might replace the values of these macros with greater values. —end note]**

Add a new paragraph to the end of [support.limits.general]:

??? A freestanding implementation shall define the `_cpp_freestanding_library` macro with the value 201803L in the `<version>` header.

Split Table 35.

The new Table ??35?? will be named **Table ??35?? — Freestanding and hosted standard library feature-test macros**. Leave the following macros in Table ??35??.

- `__cpp_lib_addressof_constexpr`
- `__cpp_lib_allocator_traits_is_always_equal`
- `__cpp_lib_apply`
- `__cpp_lib_array_constexpr`
- `__cpp_lib_as_const`
- `__cpp_lib_atomic_is_always_lock_free`
- `__cpp_lib_atomic_ref`
- `__cpp_lib_bit_cast`
- `__cpp_lib_bool_constant`
- `__cpp_lib_boyer_moore_searcher`
- `__cpp_lib_byte`
- `__cpp_lib_chrono`
- `__cpp_lib_clamp`
- `__cpp_lib_concepts`
- `__cpp_lib_constexpr_swap_algorithms`
- `__cpp_lib_exchange_function`
- `__cpp_lib_gcd_lcm`
- `__cpp_lib_hardware_interference_size`
- `__cpp_lib_has_unique_object_representations`
- `__cpp_lib_integer_sequence`
- `__cpp_lib_integral_constant_callable`
- `__cpp_lib_invoke`
- `__cpp_lib_is_aggregate`
- `__cpp_lib_is_final`
- `__cpp_lib_is_invocable`
- `__cpp_lib_is_null_pointer`
- `__cpp_lib_is_swappable`
- `__cpp_lib_laundry`
- `__cpp_lib_logical_traits`
- `__cpp_lib_make_from_tuple`
- `__cpp_lib_make_reverse_iterator`
- `__cpp_lib_nonmember_container_access`
- `__cpp_lib_not_fn`
- `__cpp_lib_null_iterators`
- `__cpp_lib_optional`
- `__cpp_lib_raw_memory_algorithms`
- `__cpp_lib_result_of_sfinae`
- `__cpp_lib_robust_nonmodifying_seq_ops`
- `__cpp_lib_sample`
- `__cpp_lib_string_view`
- `__cpp_lib_to_chars`
- `__cpp_lib_transformation_trait_aliases`
- `__cpp_lib_transparent_operators`
- `__cpp_lib_tuple_element_t`
- `__cpp_lib_tuples_by_type`
- `__cpp_lib_type_trait_variable_templates`
- `__cpp_lib_uncaught_exceptions`
- `__cpp_lib_variant`
- `__cpp_lib_void_t`

The new Table ??36?? will be named **Table ??36?? — Hosted standard library feature-test macros**. Move the following macros into Table ??36??.

- `__cpp_lib_any`
- `__cpp_lib_complex_udls`
- `__cpp_lib_enable_shared_from_this`
- `__cpp_lib_execution`
- `__cpp_lib_filesystem`
- `__cpp_lib_generic_associative_lookup`
- `__cpp_lib_hypot`
- `__cpp_lib_incomplete_container_elements`
- `__cpp_lib_list_remove_return_type`
- `__cpp_lib_make_unique`
- `__cpp_lib_map_try_emplace`
- `__cpp_lib_math_special_functions`
- `__cpp_lib_memory_resource`
- `__cpp_lib_node_extract`
- `__cpp_lib_parallel_algorithm`
- `__cpp_lib_quoted_string_io`
- `__cpp_lib_scoped_lock`
- `__cpp_lib_shared_mutex`
- `__cpp_lib_shared_ptr_arrays`
- `__cpp_lib_shared_ptr_weak_type`

- `__cpp_lib_shared_timed_mutex`
- `__cpp_lib_string_udls`
- `__cpp_lib_unordered_map_try_emplace`

Change in [cerrno.syn]:

???

Header <cerrno> synopsis

[cerrno.syn]

```
#define errno see below

#define E2BIG see below // freestanding
#define EACCES see below // freestanding
#define EADDRINUSE see below // freestanding
#define EADDRNOTAVAIL see below // freestanding
#define EAFNOSUPPORT see below // freestanding
#define EAGAIN see below // freestanding
#define EALREADY see below // freestanding
#define EBADF see below // freestanding
#define EBADMSG see below // freestanding
#define EBUSY see below // freestanding
#define ECANCELED see below // freestanding
#define ECHILD see below // freestanding
#define ECONNABORTED see below // freestanding
#define ECONNREFUSED see below // freestanding
#define ECONNRESET see below // freestanding
#define EDEADLK see below // freestanding
#define EDESTADDRREQ see below // freestanding
#define EDOM see below // freestanding
#define EEXIST see below // freestanding
#define EFAULT see below // freestanding
#define EFBIG see below // freestanding
#define EHOSTUNREACH see below // freestanding
#define EIDRM see below // freestanding
#define EILSEQ see below // freestanding
#define EINPROGRESS see below // freestanding
#define EINTR see below // freestanding
#define EINVAL see below // freestanding
#define EIO see below // freestanding
#define EISCONN see below // freestanding
#define EISDIR see below // freestanding
#define ELOOP see below // freestanding
#define EMFILE see below // freestanding
#define EMLINK see below // freestanding
#define EMSGSIZE see below // freestanding
#define ENAMETOOLONG see below // freestanding
#define ENETDOWN see below // freestanding
#define ENETRESET see below // freestanding
#define ENETUNREACH see below // freestanding
#define ENFILE see below // freestanding
#define ENOBUFS see below // freestanding
#define ENODATA see below // freestanding
#define ENODEV see below // freestanding
#define ENOENT see below // freestanding
#define ENOEXEC see below // freestanding
#define ENOLCK see below // freestanding
#define ENOLINK see below // freestanding
#define ENOMEM see below // freestanding
#define ENOMSG see below // freestanding
#define ENOPROTOOPT see below // freestanding
#define ENOSPC see below // freestanding
#define ENOSR see below // freestanding
#define ENOSTR see below // freestanding
#define ENOSYS see below // freestanding
#define ENOTCONN see below // freestanding
#define ENOTDIR see below // freestanding
#define ENOTEMPTY see below // freestanding
#define ENOTRECOVERABLE see below // freestanding
#define ENOTSOCK see below // freestanding
#define ENOTSUP see below // freestanding
```

```

#define ENOTTY see below // freestanding
#define ENXIO see below // freestanding
#define EOPNOTSUPP see below // freestanding
#define EOVERFLOW see below // freestanding
#define EOWNERDEAD see below // freestanding
#define EPERM see below // freestanding
#define EPIPE see below // freestanding
#define EPROTO see below // freestanding
#define EPROTONOSUPPORT see below // freestanding
#define EPROTOTYPE see below // freestanding
#define ERANGE see below // freestanding
#define EROFS see below // freestanding
#define ESPIPE see below // freestanding
#define ESRCH see below // freestanding
#define ETIME see below // freestanding
#define ETIMEDOUT see below // freestanding
#define ETXTBSY see below // freestanding
#define EWOULDBLOCK see below // freestanding
#define EXDEV see below // freestanding

```

Change in [system_error.syn]:

???

Header <system_error> synopsis

[system_error.syn]

```

namespace std {
    class error_category;
    const error_category& generic_category() noexcept;
    const error_category& system_category() noexcept;

    class error_code;
    class error_condition;
    class system_error;

    template<class T>
        struct is_error_code_enum : public false_type {};

    template<class T>
        struct is_error_condition_enum : public false_type {};

    enum class errc { // freestanding
        address_family_not_supported, // EAFNOSUPPORT
        address_in_use, // EADDRINUSE
        address_not_available, // EADDRNOTAVAIL
    };
}

```

Change in [optional.syn]:

???

Header <optional> synopsis

[optional.syn]

```

namespace std {
    // [optional.optional], class template optional
    template<class T>
        class optional; // freestanding, partial

    // [optional.nullopt], no-value state indicator
    struct nullopt_t{see below}; // freestanding
    inline constexpr nullopt_t nullopt(unspecified); // freestanding

    // [optional.bad.access], class bad_optional_access
    class bad_optional_access;

    // [optional.relops], relational operators
    template<class T, class U>
        constexpr bool operator==(const optional<T>&, const optional<U>&); // freestanding
    template<class T, class U>
        constexpr bool operator!=(const optional<T>&, const optional<U>&); // freestanding
    template<class T, class U>
        constexpr bool operator<(const optional<T>&, const optional<U>&); // freestanding
    template<class T, class U>

```

```

constexpr bool operator>(const optional<T>&, const optional<U>&); // freestanding
template<class T, class U>
constexpr bool operator<=(const optional<T>&, const optional<U>&); // freestanding
template<class T, class U>
constexpr bool operator>=(const optional<T>&, const optional<U>&); // freestanding

// [optional.nullops], comparison with nullopt
template<class T> constexpr bool operator==(const optional<T>&, nullopt_t) noexcept; // freestanding
template<class T> constexpr bool operator==(nullopt_t, const optional<T>&) noexcept; // freestanding
template<class T> constexpr bool operator!=(const optional<T>&, nullopt_t) noexcept; // freestanding
template<class T> constexpr bool operator!=(nullopt_t, const optional<T>&) noexcept; // freestanding
template<class T> constexpr bool operator<(const optional<T>&, nullopt_t) noexcept; // freestanding
template<class T> constexpr bool operator<(nullopt_t, const optional<T>&) noexcept; // freestanding
template<class T> constexpr bool operator>(const optional<T>&, nullopt_t) noexcept; // freestanding
template<class T> constexpr bool operator>(nullopt_t, const optional<T>&) noexcept; // freestanding
template<class T> constexpr bool operator<=(const optional<T>&, nullopt_t) noexcept; // freestanding
template<class T> constexpr bool operator<=(nullopt_t, const optional<T>&) noexcept; // freestanding
template<class T> constexpr bool operator>=(const optional<T>&, nullopt_t) noexcept; // freestanding
template<class T> constexpr bool operator>=(nullopt_t, const optional<T>&) noexcept; // freestanding

// [optional.comp_with_t], comparison with T
template<class T, class U> constexpr bool operator==(const optional<T>&, const U&); // freestanding
template<class T, class U> constexpr bool operator==(const T&, const optional<U>&); // freestanding
template<class T, class U> constexpr bool operator!=(const optional<T>&, const U&); // freestanding
template<class T, class U> constexpr bool operator!=(const T&, const optional<U>&); // freestanding
template<class T, class U> constexpr bool operator<(const optional<T>&, const U&); // freestanding
template<class T, class U> constexpr bool operator<(const T&, const optional<U>&); // freestanding
template<class T, class U> constexpr bool operator>(const optional<T>&, const U&); // freestanding
template<class T, class U> constexpr bool operator>(const T&, const optional<U>&); // freestanding
template<class T, class U> constexpr bool operator<=(const optional<T>&, const U&); // freestanding
template<class T, class U> constexpr bool operator<=(const T&, const optional<U>&); // freestanding
template<class T, class U> constexpr bool operator>=(const optional<T>&, const U&); // freestanding
template<class T, class U> constexpr bool operator>=(const T&, const optional<U>&); // freestanding

// [optional.specalg], specialized algorithms
template<class T>
    void swap(optional<T>&, optional<T>&) noexcept(see below); // freestanding

template<class T>
    constexpr optional<see below> make_optional(T&&); // freestanding
template<class T, class... Args>
    constexpr optional<T> make_optional(Args&&... args); // freestanding
template<class T, class U, class... Args>
    constexpr optional<T> make_optional(initializer_list<U> il, Args&&... args); // freestanding

// [optional.hash], hash support
template<class T> struct hash; // freestanding
template<class T> struct hash<optional<T>>; // freestanding
}

```

Change in [optional.optional]

```

// [optional.observe], observers
constexpr const T* operator->() const;
constexpr T* operator->();
constexpr const T& operator*() const&;
constexpr T& operator*() &;
constexpr T&& operator*() &&;
constexpr const T&& operator*() const&&;
constexpr explicit operator bool() const noexcept;
constexpr bool has_value() const noexcept;
constexpr const T& value() const&; // freestanding, omit
constexpr T& value() &; // freestanding, omit
constexpr T&& value() &&; // freestanding, omit
constexpr const T&& value() const&&; // freestanding, omit
template<class U> constexpr T value_or(U&&) const&;
template<class U> constexpr T value_or(U&&) &&;

```

Change in [variant.syn]:


```

namespace std {
    // [variant.variant], class template variant
    template<class... Types>
        class variant; // freestanding

    // [variant.helper], variant helper classes
    template<class T> struct variant_size; // not defined, freestanding
    template<class T> struct variant_size<const T>; // freestanding
    template<class T> struct variant_size<volatile T>; // freestanding
    template<class T> struct variant_size<const volatile T>; // freestanding
    template<class T>
        inline constexpr size_t variant_size_v = variant_size<T>::value; // freestanding

    template<class... Types>
        struct variant_size<variant<Types...>>; // freestanding

    template<size_t I, class T> struct variant_alternative; // not defined, freestanding
    template<size_t I, class T> struct variant_alternative<I, const T>; // freestanding
    template<size_t I, class T> struct variant_alternative<I, volatile T>; // freestanding
    template<size_t I, class T> struct variant_alternative<I, const volatile T>; // freestanding
    template<size_t I, class T>
        using variant_alternative_t = typename variant_alternative<I, T>::type; // freestanding

    template<size_t I, class... Types>
        struct variant_alternative<I, variant<Types...>>; // freestanding

    inline constexpr size_t variant_npos = -1; // freestanding

    // [value.get], value access
    template<class T, class... Types> // freestanding
        constexpr bool holds_alternative(const variant<Types...>&) noexcept;

    template<size_t I, class... Types> // freestanding
        constexpr variant_alternative_t<I, variant<Types...>>& get(variant<Types...>&);
    template<size_t I, class... Types> // freestanding
        constexpr variant_alternative_t<I, variant<Types...>>&& get(variant<Types...>&&);
    template<size_t I, class... Types> // freestanding
        constexpr const variant_alternative_t<I, variant<Types...>>& get(const variant<Types...>&);
    template<size_t I, class... Types> // freestanding
        constexpr const variant_alternative_t<I, variant<Types...>>&& get(const variant<Types...>&&);

    template<class T, class... Types>
        constexpr T& get(variant<Types...>&);
    template<class T, class... Types>
        constexpr T&& get(variant<Types...>&&);
    template<class T, class... Types>
        constexpr const T& get(const variant<Types...>&);
    template<class T, class... Types>
        constexpr const T&& get(const variant<Types...>&&);

    template<size_t I, class... Types>
        constexpr add_pointer_t<variant_alternative_t<I, variant<Types...>>>
            get_if(variant<Types...>*) noexcept; // freestanding
    template<size_t I, class... Types>
        constexpr add_pointer_t<const variant_alternative_t<I, variant<Types...>>>
            get_if(const variant<Types...>*) noexcept; // freestanding

    template<class T, class... Types>
        constexpr add_pointer_t<T>
            get_if(variant<Types...>*) noexcept; // freestanding
    template<class T, class... Types>
        constexpr add_pointer_t<const T>
            get_if(const variant<Types...>*) noexcept; // freestanding

    // [variant.relops], relational operators
    template<class... Types> // freestanding
        constexpr bool operator==(const variant<Types...>&, const variant<Types...>&);
    template<class... Types> // freestanding
        constexpr bool operator!=(const variant<Types...>&, const variant<Types...>&);
    template<class... Types> // freestanding
        constexpr bool operator<(const variant<Types...>&, const variant<Types...>&);

```



```

template<class... Types> // freestanding
    constexpr bool operator>(const variant<Types...>&, const variant<Types...>&);
template<class... Types> // freestanding
    constexpr bool operator<=(const variant<Types...>&, const variant<Types...>&);
template<class... Types> // freestanding
    constexpr bool operator>=(const variant<Types...>&, const variant<Types...>&);

// [variant.visit], visitation
template<class Visitor, class... Variants>
    constexpr see below visit(Visitor&&, Variants&&...); // freestanding

// [variant.monostate], class monostate
struct monostate; // freestanding

// [variant.monostate.relops], monostate relational operators
constexpr bool operator==(monostate, monostate) noexcept; // freestanding
constexpr bool operator!=(monostate, monostate) noexcept; // freestanding
constexpr bool operator<(monostate, monostate) noexcept; // freestanding
constexpr bool operator>(monostate, monostate) noexcept; // freestanding
constexpr bool operator<=(monostate, monostate) noexcept; // freestanding
constexpr bool operator>=(monostate, monostate) noexcept; // freestanding

// [variant.specalg], specialized algorithms
template<class... Types> // freestanding
    void swap(variant<Types...>&, variant<Types...>&) noexcept(see below);

// [variant.bad.access], class bad_variant_access
class bad_variant_access; // freestanding

// [variant.hash], hash support
template<class T> struct hash; // freestanding
template<class... Types> struct hash<variant<Types...>>; // freestanding
template<> struct hash<monostate>; // freestanding
}

```

Change in [bitset.syn]:

???

Header <bitset> synopsis

[bitset.syn]

```

#include <string>
#include <iosfwd> // for istream, ostream, see [iosfwd.syn]

namespace std {
    template<size_t N> class bitset; // freestanding, partial

    // [bitset.operators], bitset operators
    template<size_t N> // freestanding
        bitset<N> operator&(const bitset<N>&, const bitset<N>&) noexcept;
    template<size_t N> // freestanding
        bitset<N> operator|(const bitset<N>&, const bitset<N>&) noexcept;
    template<size_t N> // freestanding
        bitset<N> operator^(const bitset<N>&, const bitset<N>&) noexcept;
    template<class charT, class traits, size_t N>
        basic_istream<charT, traits>&
            operator>>(basic_istream<charT, traits>& is, bitset<N>& x);
    template<class charT, class traits, size_t N>
        basic_ostream<charT, traits>&
            operator<<(basic_ostream<charT, traits>& os, const bitset<N>& x);
}

```

Change in [template.bitset]:

???

Class template bitset

[template.bitset]

```

namespace std {
    template<size_t N> class bitset {
    public:
        // bit reference

```

```

class reference {
    friend class bitset;
    reference() noexcept;

public:
    reference(const reference&) = default;
    ~reference();
    reference& operator=(bool x) noexcept;           // for b[i] = x;
    reference& operator=(const reference&) noexcept; // for b[i] = b[j];
    bool operator~() const noexcept;                // flips the bit
    operator bool() const noexcept;                  // for x = b[i];
    reference& flip() noexcept;                       // for b[i].flip();
};

// [bitset.cons], constructors
constexpr bitset() noexcept;
constexpr bitset(unsigned long long val) noexcept;
template<class charT, class traits, class Allocator>
explicit bitset(
    const basic_string<charT, traits, Allocator>& str,
    typename basic_string<charT, traits, Allocator>::size_type pos = 0,
    typename basic_string<charT, traits, Allocator>::size_type n
        = basic_string<charT, traits, Allocator>::npos,
    charT zero = charT('0'),
    charT one = charT('1')); // freestanding, omit
template<class charT>
explicit bitset(
    const charT* str,
    typename basic_string<charT>::size_type n = basic_string<charT>::npos,
    charT zero = charT('0'),
    charT one = charT('1')); // freestanding, omit

// [bitset.members], bitset operations
bitset<N>& operator&=(const bitset<N>& rhs) noexcept;
bitset<N>& operator|=(const bitset<N>& rhs) noexcept;
bitset<N>& operator^=(const bitset<N>& rhs) noexcept;
bitset<N>& operator<<=(size_t pos) noexcept;
bitset<N>& operator>>=(size_t pos) noexcept;
bitset<N>& set() noexcept;
bitset<N>& set(size_t pos, bool val = true); // freestanding, omit
bitset<N>& reset() noexcept;
bitset<N>& reset(size_t pos); // freestanding, omit
bitset<N> operator~() const noexcept;
bitset<N>& flip() noexcept;
bitset<N>& flip(size_t pos); // freestanding, omit

// element access
constexpr bool operator[](size_t pos) const; // for b[i];
reference operator[](size_t pos);           // for b[i];

unsigned long to_ulong() const;
unsigned long long to_ullong() const;
template<class charT = char,
        class traits = char_traits<charT>,
        class Allocator = allocator<charT>>
    basic_string<charT, traits, Allocator>
    to_string(charT zero = charT('0'), charT one = charT('1')) const; // freestanding, omit

size_t count() const noexcept;
constexpr size_t size() const noexcept;
bool operator==(const bitset<N>& rhs) const noexcept;
bool operator!=(const bitset<N>& rhs) const noexcept;
bool test(size_t pos) const;
bool all() const noexcept;
bool any() const noexcept;
bool none() const noexcept;
bitset<N> operator<<(size_t pos) const noexcept;
bitset<N> operator>>(size_t pos) const noexcept;
};

// [bitset.hash], hash support
template<class T> struct hash;
template<size_t N> struct hash<bitset<N>>;
}

```

???

Header <memory> synopsis**[memory.syn]**

- 1 The header <memory> defines several types and function templates that describe properties of pointers and pointer-like types, manage memory for containers and other template types, destroy objects, and construct multiple objects in uninitialized memory buffers ([pointer.traits]-[specialized.algorithms]). The header also defines the templates unique_ptr, shared_ptr, weak_ptr, and various function templates that operate on objects of these types ([smartptr]).

```

namespace std {
    // [pointer.traits], pointer traits
    template<class Ptr> struct pointer_traits; // freestanding
    template<class T> struct pointer_traits<T*>; // freestanding

    // [pointer.conversion], pointer conversion
    template<class Ptr>
        auto to_address(const Ptr& p) noexcept; // freestanding
    template<class T>
        constexpr T* to_address(T* p) noexcept; // freestanding

    // [util.dynamic.safety], pointer safety
    enum class pointer_safety { relaxed, preferred, strict };
    void declare_reachable(void* p);
    template<class T>
        T* undeclare_reachable(T* p);
    void declare_no_pointers(char* p, size_t n);
    void undeclare_no_pointers(char* p, size_t n);
    pointer_safety get_pointer_safety() noexcept;

    // [ptr.align], pointer alignment function
    void* align(size_t alignment, size_t size, void*& ptr, size_t& space); // freestanding

    // [allocator.tag], allocator argument tag
    struct allocator_arg_t { explicit allocator_arg_t() = default; }; // freestanding
    inline constexpr allocator_arg_t allocator_arg{}; // freestanding

    // [allocator.uses], uses_allocator
    template<class T, class Alloc> struct uses_allocator; // freestanding

    // [allocator.traits], allocator traits
    template<class Alloc> struct allocator_traits; // freestanding

    // [default.allocator], the default allocator
    template<class T> class allocator;
    template<class T, class U>
        bool operator==(const allocator<T>&, const allocator<U>&) noexcept;
    template<class T, class U>
        bool operator!=(const allocator<T>&, const allocator<U>&) noexcept;

    // [specialized.algorithms], specialized algorithms
    template<class T>
        constexpr T* addressof(T& r) noexcept; // freestanding
    template<class T>
        const T* addressof(const T&) = delete; // freestanding
    template<class ForwardIterator>
        void uninitialized_default_construct(ForwardIterator first, ForwardIterator last); // freestanding
    template<class ExecutionPolicy, class ForwardIterator>
        void uninitialized_default_construct(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
                                           ForwardIterator first, ForwardIterator last);
    template<class ForwardIterator, class Size>
        ForwardIterator uninitialized_default_construct_n(ForwardIterator first, Size n); // freestanding
    template<class ExecutionPolicy, class ForwardIterator, class Size>
        ForwardIterator uninitialized_default_construct_n(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
                                                         ForwardIterator first, Size n);
    template<class ForwardIterator>
        void uninitialized_value_construct(ForwardIterator first, ForwardIterator last); // freestanding
    template<class ExecutionPolicy, class ForwardIterator>
        void uninitialized_value_construct(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
                                           ForwardIterator first, ForwardIterator last);
    template<class ForwardIterator, class Size>
        ForwardIterator uninitialized_value_construct_n(ForwardIterator first, Size n); // freestanding
    template<class ExecutionPolicy, class ForwardIterator, class Size>
        ForwardIterator uninitialized_value_construct_n(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]

```

```

        ForwardIterator first, Size n);
template<class InputIterator, class ForwardIterator>
    ForwardIterator uninitialized_copy(InputIterator first, InputIterator last,
        ForwardIterator result); // freestanding
template<class ExecutionPolicy, class InputIterator, class ForwardIterator>
    ForwardIterator uninitialized_copy(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
        InputIterator first, InputIterator last,
        ForwardIterator result);
template<class InputIterator, class Size, class ForwardIterator>
    ForwardIterator uninitialized_copy_n(InputIterator first, Size n,
        ForwardIterator result); // freestanding
template<class ExecutionPolicy, class InputIterator, class Size, class ForwardIterator>
    ForwardIterator uninitialized_copy_n(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
        InputIterator first, Size n,
        ForwardIterator result);
template<class InputIterator, class ForwardIterator>
    ForwardIterator uninitialized_move(InputIterator first, InputIterator last,
        ForwardIterator result); // freestanding
template<class ExecutionPolicy, class InputIterator, class ForwardIterator>
    ForwardIterator uninitialized_move(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
        InputIterator first, InputIterator last,
        ForwardIterator result);
template<class InputIterator, class Size, class ForwardIterator>
    pair<InputIterator, ForwardIterator> uninitialized_move_n(InputIterator first, Size n,
        ForwardIterator result); // freestanding
template<class ExecutionPolicy, class InputIterator, class Size, class ForwardIterator>
    pair<InputIterator, ForwardIterator> uninitialized_move_n(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
        InputIterator first, Size n,
        ForwardIterator result);

template<class ForwardIterator, class T>
    void uninitialized_fill(ForwardIterator first, ForwardIterator last, const T& x); // freestanding
template<class ExecutionPolicy, class ForwardIterator, class T>
    void uninitialized_fill(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
        ForwardIterator first, ForwardIterator last, const T& x);
template<class ForwardIterator, class Size, class T>
    ForwardIterator uninitialized_fill_n(ForwardIterator first, Size n, const T& x); // freestanding
template<class ExecutionPolicy, class ForwardIterator, class Size, class T>
    ForwardIterator uninitialized_fill_n(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
        ForwardIterator first, Size n, const T& x);

template<class T>
    void destroy_at(T* location); // freestanding
template<class ForwardIterator>
    void destroy(ForwardIterator first, ForwardIterator last); // freestanding
template<class ExecutionPolicy, class ForwardIterator>
    void destroy(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
        ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Size>
    ForwardIterator destroy_n(ForwardIterator first, Size n); // freestanding
template<class ExecutionPolicy, class ForwardIterator, class Size>
    ForwardIterator destroy_n(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
        ForwardIterator first, Size n);

// [unique.ptr], class template unique_ptr
template<class T> struct default_delete;
template<class T> struct default_delete<T[]>;
template<class T, class D = see below default_delete<T>> class unique_ptr; // freestanding
template<class T, class D> class unique_ptr<T[], D>; // freestanding

template<class T, class... Args> unique_ptr<T>
    make_unique(Args&&... args); // T is not array
template<class T> unique_ptr<T>
    make_unique(size_t n); // T is U[]
template<class T, class... Args>
    unspecified make_unique(Args&&...) = delete; // T is U[N]

template<class T, class D> // freestanding
    void swap(unique_ptr<T, D>& x, unique_ptr<T, D>& y) noexcept;

template<class T1, class D1, class T2, class D2> // freestanding
    bool operator==(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2> // freestanding
    bool operator!=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2> // freestanding
    bool operator<(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2> // freestanding

```

```

    bool operator>(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2> // freestanding
    bool operator<=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2> // freestanding
    bool operator>=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);

template<class T, class D> // freestanding
    bool operator==(const unique_ptr<T, D>& x, nullptr_t) noexcept;
template<class T, class D> // freestanding
    bool operator==(nullptr_t, const unique_ptr<T, D>& y) noexcept;
template<class T, class D> // freestanding
    bool operator!=(const unique_ptr<T, D>& x, nullptr_t) noexcept;
template<class T, class D> // freestanding
    bool operator!=(nullptr_t, const unique_ptr<T, D>& y) noexcept;
template<class T, class D> // freestanding
    bool operator<(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D> // freestanding
    bool operator<(nullptr_t, const unique_ptr<T, D>& y);
template<class T, class D> // freestanding
    bool operator>(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D> // freestanding
    bool operator>(nullptr_t, const unique_ptr<T, D>& y);
template<class T, class D> // freestanding
    bool operator<=(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D> // freestanding
    bool operator<=(nullptr_t, const unique_ptr<T, D>& y);
template<class T, class D> // freestanding
    bool operator>=(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D> // freestanding
    bool operator>=(nullptr_t, const unique_ptr<T, D>& y);

template<class E, class T, class Y, class D>
    basic_ostream<E, T>& operator<<(basic_ostream<E, T>& os, const unique_ptr<Y, D>& p);

// [util.smartptr.weak.bad], class bad_weak_ptr
class bad_weak_ptr;

// [util.smartptr.shared], class template shared_ptr
template<class T> class shared_ptr;

// [util.smartptr.shared.create], shared_ptr creation
template<class T, class... Args>
    shared_ptr<T> make_shared(Args&&... args); // T is not array
template<class T, class A, class... Args>
    shared_ptr<T> allocate_shared(const A& a, Args&&... args); // T is not array

template<class T>
    shared_ptr<T> make_shared(size_t N); // T is U[]
template<class T, class A>
    shared_ptr<T> allocate_shared(const A& a, size_t N); // T is U[]

template<class T>
    shared_ptr<T> make_shared(); // T is U[N]
template<class T, class A>
    shared_ptr<T> allocate_shared(const A& a); // T is U[N]

template<class T>
    shared_ptr<T> make_shared(size_t N, const remove_extent_t<T>& u); // T is U[]
template<class T, class A>
    shared_ptr<T> allocate_shared(const A& a, size_t N,
                                const remove_extent_t<T>& u); // T is U[]

template<class T> shared_ptr<T>
    make_shared(const remove_extent_t<T>& u); // T is U[N]
template<class T, class A>
    shared_ptr<T> allocate_shared(const A& a, const remove_extent_t<T>& u); // T is U[N]

// [util.smartptr.shared.cmp], shared_ptr comparisons
template<class T, class U>
    bool operator==(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
    bool operator!=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
    bool operator<(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>

```

```

    bool operator>(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
    bool operator<=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
    bool operator>=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;

template<class T>
    bool operator==(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
    bool operator==(nullptr_t, const shared_ptr<T>& y) noexcept;
template<class T>
    bool operator!=(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
    bool operator!=(nullptr_t, const shared_ptr<T>& y) noexcept;
template<class T>
    bool operator<(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
    bool operator<(nullptr_t, const shared_ptr<T>& y) noexcept;
template<class T>
    bool operator<=(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
    bool operator<=(nullptr_t, const shared_ptr<T>& y) noexcept;
template<class T>
    bool operator>(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
    bool operator>(nullptr_t, const shared_ptr<T>& y) noexcept;
template<class T>
    bool operator>=(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
    bool operator>=(nullptr_t, const shared_ptr<T>& y) noexcept;

// [util.smartptr.shared.spec], shared_ptr specialized algorithms
template<class T>
    void swap(shared_ptr<T>& a, shared_ptr<T>& b) noexcept;

// [util.smartptr.shared.cast], shared_ptr casts
template<class T, class U>
    shared_ptr<T> static_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
    shared_ptr<T> dynamic_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
    shared_ptr<T> const_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
    shared_ptr<T> reinterpret_pointer_cast(const shared_ptr<U>& r) noexcept;

// [util.smartptr.getdeleter], shared_ptr get_deleter
template<class D, class T>
    D* get_deleter(const shared_ptr<T>& p) noexcept;

// [util.smartptr.shared.io], shared_ptr I/O
template<class E, class T, class Y>
    basic_ostream<E, T> operator<<(basic_ostream<E, T>& os, const shared_ptr<Y>& p);

// [util.smartptr.weak], class template weak_ptr
template<class T> class weak_ptr;

// [util.smartptr.weak.spec], weak_ptr specialized algorithms
template<class T> void swap(weak_ptr<T>& a, weak_ptr<T>& b) noexcept;

// [util.smartptr.ownerless], class template owner_less
template<class T = void> struct owner_less;

// [util.smartptr.enab], class template enable_shared_from_this
template<class T> class enable_shared_from_this;

// [util.smartptr.hash], hash support
template<class T> struct hash; // freestanding
template<class T, class D> struct hash<unique_ptr<T, D>>; // freestanding
template<class T> struct hash<shared_ptr<T>>;

// [util.smartptr.atomic], atomic smart pointers
template<class T> struct atomic<shared_ptr<T>>;
template<class T> struct atomic<weak_ptr<T>>;

// [allocator.uses.trait], uses_allocator
template<class T, class Alloc>

```

```

    inline constexpr bool uses_allocator_v = uses_allocator<T, Alloc>::value; // freestanding
}

```

Change in [unique.ptr.dltr.general] paragraph 1:

The class template `default_delete` serves as the default deleter (destruction policy) for the class template `unique_ptr` **for hosted implementations.**

Change in [unique.ptr.single]:

??? **unique_ptr for single objects** [unique.ptr.single]

```

namespace std {
    template<class T, class D = see belowdefault_delete<T>> class unique_ptr {
    public:
        using pointer      = see below;
        using element_type = T;
        using deleter_type = D;

        // [unique.ptr.single.ctor], constructors
        constexpr unique_ptr() noexcept;
        explicit unique_ptr(pointer p) noexcept;
        unique_ptr(pointer p, see below d1) noexcept;
        unique_ptr(pointer p, see below d2) noexcept;
        unique_ptr(unique_ptr&& u) noexcept;
        constexpr unique_ptr(nullptr_t) noexcept;
        template<class U, class E>
            unique_ptr(unique_ptr<U, E>&& u) noexcept;

        // [unique.ptr.single.dtor], destructor
        ~unique_ptr();

        // [unique.ptr.single.asgn], assignment
        unique_ptr& operator=(unique_ptr&& u) noexcept;
        template<class U, class E>
            unique_ptr& operator=(unique_ptr<U, E>&& u) noexcept;
        unique_ptr& operator=(nullptr_t) noexcept;

        // [unique.ptr.single.observers], observers
        add_lvalue_reference_t<T> operator*() const;
        pointer operator->() const noexcept;
        pointer get() const noexcept;
        deleter_type& get_deleter() noexcept;
        const deleter_type& get_deleter() const noexcept;
        explicit operator bool() const noexcept;

        // [unique.ptr.single.modifiers], modifiers
        pointer release() noexcept;
        void reset(pointer p = pointer()) noexcept;
        void swap(unique_ptr& u) noexcept;

        // disable copy from lvalue
        unique_ptr(const unique_ptr&) = delete;
        unique_ptr& operator=(const unique_ptr&) = delete;
    };
}

```

- ¹ The default type for the template parameter `D` is `default_delete` **for hosted implementations.** **The default type for the template parameter `D` is implementation defined for freestanding implementations.** **If the type of the template argument `D` matches the default type for the template parameter `D`, and that type is not `default_delete`, then the program is ill-formed.** [Footnote: The intent is to require freestanding users to provide a deleter argument, rather than rely on `default_delete`, as `default_delete` may not be present in a freestanding implementation.] A client-supplied template argument `D` shall be a function object type, lvalue reference to function, or lvalue reference to function object type for which, given a value `d` of type `D` and a value `ptr` of type `unique_ptr<T, D>::pointer`, the expression `d(ptr)` is valid and has the effect of disposing of the pointer as appropriate for that deleter.

Change in [functional.syn]:

??? **Header <functional> synopsis** [functional.syn]


```

namespace std {
    // [func.invoke], invoke
    template<class F, class... Args>
        invoke_result_t<F, Args...> invoke(F&& f, Args&&... args)
            noexcept(is_nothrow_invocable_v<F, Args...>); // freestanding

    // [refwrap], reference_wrapper
    template<class T> class reference_wrapper; // freestanding

    template<class T> reference_wrapper<T> ref(T&) noexcept; // freestanding
    template<class T> reference_wrapper<const T> cref(const T&) noexcept; // freestanding
    template<class T> void ref(const T&) = delete; // freestanding
    template<class T> void cref(const T&) = delete; // freestanding

    template<class T> reference_wrapper<T> ref(reference_wrapper<T>) noexcept; // freestanding
    template<class T> reference_wrapper<const T> cref(reference_wrapper<T>) noexcept; // freestanding

    // [arithmetic.operations], arithmetic operations
    template<class T = void> struct plus; // freestanding
    template<class T = void> struct minus; // freestanding
    template<class T = void> struct multiplies; // freestanding
    template<class T = void> struct divides; // freestanding
    template<class T = void> struct modulus; // freestanding
    template<class T = void> struct negate; // freestanding
    template<> struct plus<void>; // freestanding
    template<> struct minus<void>; // freestanding
    template<> struct multiplies<void>; // freestanding
    template<> struct divides<void>; // freestanding
    template<> struct modulus<void>; // freestanding
    template<> struct negate<void>; // freestanding

    // [comparisons], comparisons
    template<class T = void> struct equal_to; // freestanding
    template<class T = void> struct not_equal_to; // freestanding
    template<class T = void> struct greater; // freestanding
    template<class T = void> struct less; // freestanding
    template<class T = void> struct greater_equal; // freestanding
    template<class T = void> struct less_equal; // freestanding
    template<> struct equal_to<void>; // freestanding
    template<> struct not_equal_to<void>; // freestanding
    template<> struct greater<void>; // freestanding
    template<> struct less<void>; // freestanding
    template<> struct greater_equal<void>; // freestanding
    template<> struct less_equal<void>; // freestanding

    // [logical.operations], logical operations
    template<class T = void> struct logical_and; // freestanding
    template<class T = void> struct logical_or; // freestanding
    template<class T = void> struct logical_not; // freestanding
    template<> struct logical_and<void>; // freestanding
    template<> struct logical_or<void>; // freestanding
    template<> struct logical_not<void>; // freestanding

    // [bitwise.operations], bitwise operations
    template<class T = void> struct bit_and; // freestanding
    template<class T = void> struct bit_or; // freestanding
    template<class T = void> struct bit_xor; // freestanding
    template<class T = void> struct bit_not; // freestanding
    template<> struct bit_and<void>; // freestanding
    template<> struct bit_or<void>; // freestanding
    template<> struct bit_xor<void>; // freestanding
    template<> struct bit_not<void>; // freestanding

    // [func.identity], identity
    struct identity; // freestanding

    // [func.not_fn], function template not_fn
    template<class F> unspecified not_fn(F&& f); // freestanding

    // [func.bind], bind
    template<class T> struct is_bind_expression; // freestanding

```



```

template<class T> struct is_placeholder; // freestanding

template<class F, class... BoundArgs>
    unspecified bind(F&&, BoundArgs&&...); // freestanding
template<class R, class F, class... BoundArgs>
    unspecified bind(F&&, BoundArgs&&...); // freestanding

namespace placeholders {
    // M is the implementation-defined number of placeholders
    see below _1; // freestanding
    see below _2; // freestanding
    .
    .
    .
    see below _M; // freestanding
}

// [func.memfn], member function adaptors
template<class R, class T>
    unspecified mem_fn(R T::*) noexcept; // freestanding

// [func.wrap], polymorphic function wrappers
class bad_function_call;

template<class> class function; // not defined
template<class R, class... ArgTypes> class function<R(ArgTypes...)>;

template<class R, class... ArgTypes>
    void swap(function<R(ArgTypes...)>&, function<R(ArgTypes...)>&) noexcept;

template<class R, class... ArgTypes>
    bool operator==(const function<R(ArgTypes...)>&, nullptr_t) noexcept;
template<class R, class... ArgTypes>
    bool operator==(nullptr_t, const function<R(ArgTypes...)>&) noexcept;
template<class R, class... ArgTypes>
    bool operator!=(const function<R(ArgTypes...)>&, nullptr_t) noexcept;
template<class R, class... ArgTypes>
    bool operator!=(nullptr_t, const function<R(ArgTypes...)>&) noexcept;

// [func.search], searchers
template<class ForwardIterator, class BinaryPredicate = equal_to<>>
    class default_searcher; // freestanding

template<class RandomAccessIterator,
        class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
        class BinaryPredicate = equal_to<>>
    class boyer_moore_searcher;

template<class RandomAccessIterator,
        class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
        class BinaryPredicate = equal_to<>>
    class boyer_moore_horspool_searcher;

// [unord.hash], hash function primary template
template<class T>
    struct hash; // freestanding

// [func.bind], function object binders
template<class T>
    inline constexpr bool is_bind_expression_v = is_bind_expression<T>::value; // freestanding
template<class T>
    inline constexpr int is_placeholder_v = is_placeholder<T>::value; // freestanding
}

```

Change in [charconv.syn]:

???

Header <charconv> synopsis

[charconv.syn]

```

namespace std {
    // floating-point format for primitive numerical conversion
    enum class chars_format {
        scientific = unspecified,
        fixed = unspecified,

```

```

    hex = unspecified,
    general = fixed | scientific
};

// [charconv.to.chars], primitive numerical output conversion
struct to_chars_result {
    char* ptr;
    errc ec;
}; // freestanding

to_chars_result to_chars(char* first, char* last, see below value, int base = 10); // freestanding

to_chars_result to_chars(char* first, char* last, float value);
to_chars_result to_chars(char* first, char* last, double value);
to_chars_result to_chars(char* first, char* last, long double value);

to_chars_result to_chars(char* first, char* last, float value, chars_format fmt);
to_chars_result to_chars(char* first, char* last, double value, chars_format fmt);
to_chars_result to_chars(char* first, char* last, long double value, chars_format fmt);

to_chars_result to_chars(char* first, char* last, float value,
                          chars_format fmt, int precision);
to_chars_result to_chars(char* first, char* last, double value,
                          chars_format fmt, int precision);
to_chars_result to_chars(char* first, char* last, long double value,
                          chars_format fmt, int precision);

// [charconv.from.chars], primitive numerical input conversion
struct from_chars_result {
    const char* ptr;
    errc ec;
}; // freestanding

from_chars_result from_chars(const char* first, const char* last,
                             see below value, int base = 10); // freestanding

from_chars_result from_chars(const char* first, const char* last, float& value,
                             chars_format fmt = chars_format::general);
from_chars_result from_chars(const char* first, const char* last, double& value,
                             chars_format fmt = chars_format::general);
from_chars_result from_chars(const char* first, const char* last, long double& value,
                             chars_format fmt = chars_format::general);
}

```

Change 1 of 2 in [time.syn]:

??

Header <chrono> synopsis

[time.syn]

```

namespace std {
    namespace chrono {
        // [time.duration], class template duration
        template<class Rep, class Period = ratio<1>> class duration; // freestanding

        // [time.point], class template time_point
        template<class Clock, class Duration = typename Clock::duration> class time_point; // freestanding
    }

    // [time.traits.specializations], common_type specializations
    template<class Rep1, class Period1, class Rep2, class Period2>
        struct common_type<chrono::duration<Rep1, Period1>,
                           chrono::duration<Rep2, Period2>>; // freestanding

    template<class Clock, class Duration1, class Duration2>
        struct common_type<chrono::time_point<Clock, Duration1>,
                           chrono::time_point<Clock, Duration2>>; // freestanding

    namespace chrono {
        // [time.traits], customization traits
        template<class Rep> struct treat_as_floating_point; // freestanding
        template<class Rep> struct duration_values; // freestanding
        template<class Rep> // freestanding
            inline constexpr bool treat_as_floating_point_v = treat_as_floating_point<Rep>::value;
    }
}

```

```

template<class T> struct is_clock;
template<class T> inline constexpr bool is_clock_v = is_clock<T>::value;

// [time.duration.nonmember], duration arithmetic
template<class Rep1, class Period1, class Rep2, class Period2>
    constexpr common_type_t<duration<Rep1, Period1>, duration<Rep2, Period2>> // freestanding
        operator+(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
template<class Rep1, class Period1, class Rep2, class Period2>
    constexpr common_type_t<duration<Rep1, Period1>, duration<Rep2, Period2>> // freestanding
        operator-(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
template<class Rep1, class Rep2>
    constexpr duration<common_type_t<Rep1, Rep2>, Period> // freestanding
        operator*(const duration<Rep1, Period>& d, const Rep2& s);
template<class Rep1, class Rep2, class Period>
    constexpr duration<common_type_t<Rep1, Rep2>, Period> // freestanding
        operator*(const Rep1& s, const duration<Rep2, Period>& d);
template<class Rep1, class Period, class Rep2>
    constexpr duration<common_type_t<Rep1, Rep2>, Period> // freestanding
        operator/(const duration<Rep1, Period>& d, const Rep2& s);
template<class Rep1, class Period1, class Rep2, class Period2>
    constexpr common_type_t<Rep1, Rep2> // freestanding
        operator/(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
template<class Rep1, class Period, class Rep2>
    constexpr duration<common_type_t<Rep1, Rep2>, Period> // freestanding
        operator%(const duration<Rep1, Period>& d, const Rep2& s);
template<class Rep1, class Period1, class Rep2, class Period2>
    constexpr common_type_t<duration<Rep1, Period1>, duration<Rep2, Period2>> // freestanding
        operator%(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);

// [time.duration.comparisons], duration comparisons
template<class Rep1, class Period1, class Rep2, class Period2>
    constexpr bool operator==(const duration<Rep1, Period1>& lhs,
                             const duration<Rep2, Period2>& rhs); // freestanding
template<class Rep1, class Period1, class Rep2, class Period2>
    constexpr bool operator!=(const duration<Rep1, Period1>& lhs,
                              const duration<Rep2, Period2>& rhs); // freestanding
template<class Rep1, class Period1, class Rep2, class Period2>
    constexpr bool operator< (const duration<Rep1, Period1>& lhs,
                             const duration<Rep2, Period2>& rhs); // freestanding
template<class Rep1, class Period1, class Rep2, class Period2>
    constexpr bool operator> (const duration<Rep1, Period1>& lhs,
                              const duration<Rep2, Period2>& rhs); // freestanding
template<class Rep1, class Period1, class Rep2, class Period2>
    constexpr bool operator<= (const duration<Rep1, Period1>& lhs,
                               const duration<Rep2, Period2>& rhs); // freestanding
template<class Rep1, class Period1, class Rep2, class Period2>
    constexpr bool operator>= (const duration<Rep1, Period1>& lhs,
                                const duration<Rep2, Period2>& rhs); // freestanding

// [time.duration.cast], duration_cast
template<class ToDuration, class Rep, class Period> // freestanding
    constexpr ToDuration duration_cast(const duration<Rep, Period>& d);
template<class ToDuration, class Rep, class Period> // freestanding
    constexpr ToDuration floor(const duration<Rep, Period>& d);
template<class ToDuration, class Rep, class Period> // freestanding
    constexpr ToDuration ceil(const duration<Rep, Period>& d);
template<class ToDuration, class Rep, class Period> // freestanding
    constexpr ToDuration round(const duration<Rep, Period>& d);

// [time.duration.io], duration I/O
template<class charT, class traits, class Rep, class Period>
    basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os,
                  const duration<Rep, Period>& d);
template<class charT, class traits, class Rep, class Period>
    basic_ostream<charT, traits>&
        to_stream(basic_ostream<charT, traits>& os, const charT* fmt,
                  const duration<Rep, Period>& d);
template<class charT, class traits, class Rep, class Period, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
        from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                   duration<Rep, Period>& d,
                   basic_string<charT, traits, Alloc>* abbrev = nullptr,
                   minutes* offset = nullptr);

// convenience typedefs

```

```

using nanoseconds = duration<signed integer type of at least 64 bits, nano>; // freestanding
using microseconds = duration<signed integer type of at least 55 bits, micro>; // freestanding
using milliseconds = duration<signed integer type of at least 45 bits, milli>; // freestanding
using seconds = duration<signed integer type of at least 35 bits>; // freestanding
using minutes = duration<signed integer type of at least 29 bits, ratio< 60>>; // freestanding
using hours = duration<signed integer type of at least 23 bits, ratio<3600>>; // freestanding
using days = duration<signed integer type of at least 25 bits,
    ratio_multiply<ratio<24>, hours::period>>; // freestanding
using weeks = duration<signed integer type of at least 22 bits,
    ratio_multiply<ratio<7>, days::period>>; // freestanding
using years = duration<signed integer type of at least 17 bits,
    ratio_multiply<ratio<146097, 400>, days::period>>; // freestanding
using months = duration<signed integer type of at least 20 bits,
    ratio_divide<years::period, ratio<12>>>; // freestanding

// [time.point.nonmember], time_point arithmetic
template<class Clock, class Duration1, class Rep2, class Period2> // freestanding
    constexpr time_point<Clock, common_type_t<Duration1, duration<Rep2, Period2>>>
        operator+(const time_point<Clock, Duration1>& lhs, const duration<Rep2, Period2>& rhs);
template<class Rep1, class Period1, class Clock, class Duration2> // freestanding
    constexpr time_point<Clock, common_type_t<duration<Rep1, Period1>, Duration2>>
        operator+(const duration<Rep1, Period1>& lhs, const time_point<Clock, Duration2>& rhs);
template<class Clock, class Duration1, class Rep2, class Period2> // freestanding
    constexpr time_point<Clock, common_type_t<Duration1, duration<Rep2, Period2>>>
        operator-(const time_point<Clock, Duration1>& lhs, const duration<Rep2, Period2>& rhs);
template<class Clock, class Duration1, class Duration2> // freestanding
    constexpr common_type_t<Duration1, Duration2>
        operator-(const time_point<Clock, Duration1>& lhs,
            const time_point<Clock, Duration2>& rhs);

// [time.point.comparisons], time_point comparisons
template<class Clock, class Duration1, class Duration2> // freestanding
    constexpr bool operator==(const time_point<Clock, Duration1>& lhs,
        const time_point<Clock, Duration2>& rhs);
template<class Clock, class Duration1, class Duration2> // freestanding
    constexpr bool operator!=(const time_point<Clock, Duration1>& lhs,
        const time_point<Clock, Duration2>& rhs);
template<class Clock, class Duration1, class Duration2> // freestanding
    constexpr bool operator< (const time_point<Clock, Duration1>& lhs,
        const time_point<Clock, Duration2>& rhs);
template<class Clock, class Duration1, class Duration2> // freestanding
    constexpr bool operator> (const time_point<Clock, Duration1>& lhs,
        const time_point<Clock, Duration2>& rhs);
template<class Clock, class Duration1, class Duration2> // freestanding
    constexpr bool operator<= (const time_point<Clock, Duration1>& lhs,
        const time_point<Clock, Duration2>& rhs);
template<class Clock, class Duration1, class Duration2> // freestanding
    constexpr bool operator>= (const time_point<Clock, Duration1>& lhs,
        const time_point<Clock, Duration2>& rhs);

// [time.point.cast], time_point_cast
template<class ToDuration, class Clock, class Duration> // freestanding
    constexpr time_point<Clock, ToDuration>
        time_point_cast(const time_point<Clock, Duration>& t);
template<class ToDuration, class Clock, class Duration> // freestanding
    constexpr time_point<Clock, ToDuration> floor(const time_point<Clock, Duration>& tp);
template<class ToDuration, class Clock, class Duration> // freestanding
    constexpr time_point<Clock, ToDuration> ceil(const time_point<Clock, Duration>& tp);
template<class ToDuration, class Clock, class Duration> // freestanding
    constexpr time_point<Clock, ToDuration> round(const time_point<Clock, Duration>& tp);

// [time.duration.alg], specialized algorithms
template<class Rep, class Period>
    constexpr duration<Rep, Period> abs(duration<Rep, Period> d); // freestanding

```

Change 2 of 2 in [time.syn]:

```

inline namespace literals {
inline namespace chrono_literals {
    // [time.duration.literals], suffixes for duration literals
    constexpr chrono::hours operator""h(unsigned long long); // freestanding
    constexpr chrono::duration<unspecified, ratio<3600, 1>> operator""h(long double); // freestanding

```

```

constexpr chrono::minutes operator""min(unsigned long long); // freestanding
constexpr chrono::duration<unspecified, ratio<60, 1>> operator""min(long double); // freestanding

constexpr chrono::seconds operator""s(unsigned long long); // freestanding
constexpr chrono::duration<unspecified> operator""s(long double); // freestanding

constexpr chrono::milliseconds operator""ms(unsigned long long); // freestanding
constexpr chrono::duration<unspecified, milli> operator""ms(long double); // freestanding

constexpr chrono::microseconds operator""us(unsigned long long); // freestanding
constexpr chrono::duration<unspecified, micro> operator""us(long double); // freestanding

constexpr chrono::nanoseconds operator""ns(unsigned long long); // freestanding
constexpr chrono::duration<unspecified, nano> operator""ns(long double); // freestanding

// [time.cal.day.nonmembers], non-member functions
constexpr chrono::day operator""d(unsigned long long d) noexcept; // freestanding

// [time.cal.year.nonmembers], non-member functions
constexpr chrono::year operator""y(unsigned long long y) noexcept; // freestanding
}
}

namespace chrono {
    using namespace literals::chrono_literals; // freestanding
}
}

```

Change in [string.syn]:

???

Header <string> synopsis

[string.syn]

```

#include <initializer_list>

namespace std {
    // [char.traits], character traits
    template<class charT> struct char_traits; // freestanding
    template<> struct char_traits<char>; // freestanding
    template<> struct char_traits<char16_t>; // freestanding
    template<> struct char_traits<char32_t>; // freestanding
    template<> struct char_traits<wchar_t>; // freestanding

```

Change in [cstring.syn]:

???

Header <cstring> synopsis

[cstring.syn]

```

namespace std {
    using size_t = see [support.types.layout]; // freestanding

    void* memcpy(void* s1, const void* s2, size_t n); // freestanding
    void* memmove(void* s1, const void* s2, size_t n); // freestanding
    char* strcpy(char* s1, const char* s2); // freestanding
    char* strncpy(char* s1, const char* s2, size_t n); // freestanding
    char* strcat(char* s1, const char* s2); // freestanding
    char* strncat(char* s1, const char* s2, size_t n); // freestanding
    int memcmp(const void* s1, const void* s2, size_t n); // freestanding
    int strcmp(const char* s1, const char* s2); // freestanding
    int strcoll(const char* s1, const char* s2);
    int strncmp(const char* s1, const char* s2, size_t n); // freestanding
    size_t strxfrm(char* s1, const char* s2, size_t n);
    const void* memchr(const void* s, int c, size_t n); // see [library.c], freestanding
    void* memchr(void* s, int c, size_t n); // see [library.c], freestanding
    const char* strchr(const char* s, int c); // see [library.c], freestanding
    char* strchr(char* s, int c); // see [library.c], freestanding
    size_t strcspn(const char* s1, const char* s2); // freestanding
    const char* strpbrk(const char* s1, const char* s2); // see [library.c], freestanding

```

```

char* strpbrk(char* s1, const char* s2); // see [library.c], freestanding
const char* strrchr(const char* s, int c); // see [library.c], freestanding
char* strrchr(char* s, int c); // see [library.c], freestanding
size_t strspn(const char* s1, const char* s2); // freestanding
const char* strstr(const char* s1, const char* s2); // see [library.c], freestanding
char* strstr(char* s1, const char* s2); // see [library.c], freestanding
char* strtok(char* s1, const char* s2);
void* memset(void* s, int c, size_t n); // freestanding
char* strerror(int errnum);
size_t strlen(const char* s); // freestanding
}

#define NULL see [support.types.nullptr] // freestanding

```

Change in [string.view.synop]:

???

Header <string_view> synopsis

[string.view.synop]

```

namespace std {
    // [string.view.template], class template basic_string_view
    template<class charT, class traits = char_traits<charT>>
    class basic_string_view; // freestanding, partial

    // [string.view.comparison], non-member comparison functions
    template<class charT, class traits>
        constexpr bool operator==(basic_string_view<charT, traits> x,
                                   basic_string_view<charT, traits> y) noexcept; // freestanding
    template<class charT, class traits>
        constexpr bool operator!=(basic_string_view<charT, traits> x,
                                   basic_string_view<charT, traits> y) noexcept; // freestanding
    template<class charT, class traits>
        constexpr bool operator< (basic_string_view<charT, traits> x,
                                   basic_string_view<charT, traits> y) noexcept; // freestanding
    template<class charT, class traits>
        constexpr bool operator> (basic_string_view<charT, traits> x,
                                   basic_string_view<charT, traits> y) noexcept; // freestanding
    template<class charT, class traits>
        constexpr bool operator<= (basic_string_view<charT, traits> x,
                                    basic_string_view<charT, traits> y) noexcept; // freestanding
    template<class charT, class traits>
        constexpr bool operator>= (basic_string_view<charT, traits> x,
                                    basic_string_view<charT, traits> y) noexcept; // freestanding
    // see [string.view.comparison], sufficient additional overloads of comparison functions, freestanding

    // [string.view.io], inserters and extractors
    template<class charT, class traits>
        basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os,
                   basic_string_view<charT, traits> str);

    // basic_string_view typedef names
    using string_view = basic_string_view<char>; // freestanding
    using u16string_view = basic_string_view<char16_t>; // freestanding
    using u32string_view = basic_string_view<char32_t>; // freestanding
    using wstring_view = basic_string_view<wchar_t>; // freestanding

    // [string.view.hash], hash support
    template<class T> struct hash; // freestanding
    template<> struct hash<string_view>; // freestanding
    template<> struct hash<u16string_view>; // freestanding
    template<> struct hash<u32string_view>; // freestanding
    template<> struct hash<wstring_view>; // freestanding

    inline namespace literals {
        inline namespace string_view_literals {
            // [string.view.literals], suffix for basic_string_view literals
            constexpr string_view operator""sv(const char* str, size_t len) noexcept; // freestanding
            constexpr u16string_view operator""sv(const char16_t* str, size_t len) noexcept; // freestanding
            constexpr u32string_view operator""sv(const char32_t* str, size_t len) noexcept; // freestanding
            constexpr wstring_view operator""sv(const wchar_t* str, size_t len) noexcept; // freestanding
        }
    }
}

```



```
}
}
```

Change in [string.view.template]:

```
// [string.view.access], element access
constexpr const_reference operator[](size_type pos) const;
constexpr const_reference at(size_type pos) const; // freestanding, omit
constexpr const_reference front() const;
constexpr const_reference back() const;
constexpr const_pointer data() const noexcept;

// [string.view.modifiers], modifiers
constexpr void remove_prefix(size_type n);
constexpr void remove_suffix(size_type n);
constexpr void swap(basic_string_view& s) noexcept;

// [string.view.ops], string operations
size_type copy(charT* s, size_type n, size_type pos = 0) const; // freestanding, omit

constexpr basic_string_view substr(size_type pos = 0, size_type n = npos) const; // freestanding, omit

constexpr int compare(basic_string_view s) const noexcept;
constexpr int compare(size_type pos1, size_type n1, basic_string_view s) const; // freestanding, omit
constexpr int compare(size_type pos1, size_type n1, basic_string_view s,
                      size_type pos2, size_type n2) const; // freestanding, omit
constexpr int compare(const charT* s) const;
constexpr int compare(size_type pos1, size_type n1, const charT* s) const; // freestanding, omit
constexpr int compare(size_type pos1, size_type n1, const charT* s, size_type n2) const; // freestanding, omit
```

Change in [cwchar.syn]:

???

Header <cwchar> synopsis

[cwchar.syn]

```
namespace std {
    using size_t = see [support.types.layout]; // freestanding
    using mbstate_t = see below; // freestanding
    using wint_t = see below; // freestanding

    struct tm;

    int fwprintf(FILE* stream, const wchar_t* format, ...);
    int fwscanf(FILE* stream, const wchar_t* format, ...);
    int swprintf(wchar_t* s, size_t n, const wchar_t* format, ...);
    int swscanf(const wchar_t* s, const wchar_t* format, ...);
    int vfwprintf(FILE* stream, const wchar_t* format, va_list arg);
    int vfwscanf(FILE* stream, const wchar_t* format, va_list arg);
    int vswprintf(wchar_t* s, size_t n, const wchar_t* format, va_list arg);
    int vswscanf(const wchar_t* s, const wchar_t* format, va_list arg);
    int vwprintf(const wchar_t* format, va_list arg);
    int vwscanf(const wchar_t* format, va_list arg);
    int wprintf(const wchar_t* format, ...);
    int wscanf(const wchar_t* format, ...);
    wint_t fgetwc(FILE* stream);
    wchar_t* fgetws(wchar_t* s, int n, FILE* stream);
    wint_t fputwc(wchar_t c, FILE* stream);
    int fputws(const wchar_t* s, FILE* stream);
    int fwide(FILE* stream, int mode);
    wint_t getwc(FILE* stream);
    wint_t getwchar();
    wint_t putwc(wchar_t c, FILE* stream);
    wint_t putwchar(wchar_t c);
    wint_t ungetwc(wint_t c, FILE* stream);
    double wcstod(const wchar_t* nptr, wchar_t** endptr);
    float wcstof(const wchar_t* nptr, wchar_t** endptr);
    long double wcstold(const wchar_t* nptr, wchar_t** endptr);
    long int wcstol(const wchar_t* nptr, wchar_t** endptr, int base);
    long long int wcstoll(const wchar_t* nptr, wchar_t** endptr, int base);
    unsigned long int wcstoul(const wchar_t* nptr, wchar_t** endptr, int base);
    unsigned long long int wcstoull(const wchar_t* nptr, wchar_t** endptr, int base);
    wchar_t* wcsncpy(wchar_t* s1, const wchar_t* s2); // freestanding
```

```

wchar_t* wcsncpy(wchar_t* s1, const wchar_t* s2, size_t n); // freestanding
wchar_t* wmemcpy(wchar_t* s1, const wchar_t* s2, size_t n); // freestanding
wchar_t* wmemmove(wchar_t* s1, const wchar_t* s2, size_t n); // freestanding
wchar_t* wscat(wchar_t* s1, const wchar_t* s2); // freestanding
wchar_t* wcsncat(wchar_t* s1, const wchar_t* s2, size_t n); // freestanding
int wcsncmp(const wchar_t* s1, const wchar_t* s2); // freestanding
int wscoll(const wchar_t* s1, const wchar_t* s2);
int wcsncmp(const wchar_t* s1, const wchar_t* s2, size_t n); // freestanding
size_t wcsxfrm(wchar_t* s1, const wchar_t* s2, size_t n);
int wmemcmp(const wchar_t* s1, const wchar_t* s2, size_t n); // freestanding
const wchar_t* wcschr(const wchar_t* s, wchar_t c); // see [library.c], freestanding
wchar_t* wcschr(wchar_t* s, wchar_t c); // see [library.c], freestanding
size_t wcsnspn(const wchar_t* s1, const wchar_t* s2); // freestanding
const wchar_t* wcpbrk(const wchar_t* s1, const wchar_t* s2); // see [library.c], freestanding
wchar_t* wcpbrk(wchar_t* s1, const wchar_t* s2); // see [library.c], freestanding
const wchar_t* wcsrchr(const wchar_t* s, wchar_t c); // see [library.c], freestanding
wchar_t* wcsrchr(wchar_t* s, wchar_t c); // see [library.c], freestanding
size_t wcsspncpy(const wchar_t* s1, const wchar_t* s2); // freestanding
const wchar_t* wcsstr(const wchar_t* s1, const wchar_t* s2); // see [library.c], freestanding
wchar_t* wcsstr(wchar_t* s1, const wchar_t* s2); // see [library.c], freestanding
wchar_t* wcstok(wchar_t* s1, const wchar_t* s2, wchar_t** ptr); // freestanding
const wchar_t* wmemchr(const wchar_t* s, wchar_t c, size_t n); // see [library.c], freestanding
wchar_t* wmemchr(wchar_t* s, wchar_t c, size_t n); // see [library.c], freestanding
size_t wcslen(const wchar_t* s); // freestanding
wchar_t* wmemset(wchar_t* s, wchar_t c, size_t n); // freestanding
size_t wcsftime(wchar_t* s, size_t maxsize, const wchar_t* format, const struct tm* timeptr);
wint_t btowc(int c);
int wctob(wint_t c);

// [c.mb.wcs], multibyte / wide string and character conversion functions
int mbsinit(const mbstate_t* ps);
size_t mbrlen(const char* s, size_t n, mbstate_t* ps);
size_t mbrtowc(wchar_t* pwc, const char* s, size_t n, mbstate_t* ps);
size_t wctomb(char* s, wchar_t wc, mbstate_t* ps);
size_t mbsrtowcs(wchar_t* dst, const char** src, size_t len, mbstate_t* ps);
size_t wcsrtombs(char* dst, const wchar_t** src, size_t len, mbstate_t* ps);
}

#define NULL see [support.types.nullptr] // freestanding
#define WCHAR_MAX see below // freestanding
#define WCHAR_MIN see below // freestanding
#define WEOF see below // freestanding

```

Change in [array.syn]:

???

Header <array> synopsis

[array.syn]

```

#include <initializer_list>

namespace std {
    // [array], class template array
    template<class T, size_t N> struct array; // freestanding, partial

    template<class T, size_t N> // freestanding
        constexpr bool operator==(const array<T, N>& x, const array<T, N>& y);
    template<class T, size_t N> // freestanding
        constexpr bool operator!=(const array<T, N>& x, const array<T, N>& y);
    template<class T, size_t N> // freestanding
        constexpr bool operator< (const array<T, N>& x, const array<T, N>& y);
    template<class T, size_t N> // freestanding
        constexpr bool operator> (const array<T, N>& x, const array<T, N>& y);
    template<class T, size_t N> // freestanding
        constexpr bool operator<= (const array<T, N>& x, const array<T, N>& y);
    template<class T, size_t N> // freestanding
        constexpr bool operator>= (const array<T, N>& x, const array<T, N>& y);
    template<class T, size_t N> // freestanding
        void swap(array<T, N>& x, array<T, N>& y) noexcept(noexcept(x.swap(y)));

    template<class T> class tuple_size; // freestanding
    template<size_t I, class T> class tuple_element; // freestanding

```



```

template<class T, size_t N>
    struct tuple_size<array<T, N>>; // freestanding
template<size_t I, class T, size_t N>
    struct tuple_element<I, array<T, N>>; // freestanding
template<size_t I, class T, size_t N>
    constexpr T& get(array<T, N>&) noexcept; // freestanding
template<size_t I, class T, size_t N>
    constexpr T&& get(array<T, N>&&) noexcept; // freestanding
template<size_t I, class T, size_t N>
    constexpr const T& get(const array<T, N>&) noexcept; // freestanding
template<size_t I, class T, size_t N>
    constexpr const T&& get(const array<T, N>&&) noexcept; // freestanding
}

```

Change in [array.overview]:

```

// element access
constexpr reference      operator[](size_type n);
constexpr const_reference operator[](size_type n) const;
constexpr reference      at(size_type n); // freestanding, omit
constexpr const_reference at(size_type n) const; // freestanding, omit
constexpr reference      front();
constexpr const_reference front() const;
constexpr reference      back();
constexpr const_reference back() const;

```

Change in [iterator.synopsis]:

Header <iterator> synopsis

[iterator.synopsis]

```

namespace std {
    // [iterator.primitives], primitives
    template<class Iterator> struct iterator_traits; // freestanding
    template<class T> struct iterator_traits<T*>; // freestanding

    struct input_iterator_tag { }; // freestanding
    struct output_iterator_tag { }; // freestanding
    struct forward_iterator_tag: public input_iterator_tag { }; // freestanding
    struct bidirectional_iterator_tag: public forward_iterator_tag { }; // freestanding
    struct random_access_iterator_tag: public bidirectional_iterator_tag { }; // freestanding

    // [iterator.operations], iterator operations
    template<class InputIterator, class Distance>
        constexpr void
            advance(InputIterator& i, Distance n); // freestanding
    template<class InputIterator>
        constexpr typename iterator_traits<InputIterator>::difference_type
            distance(InputIterator first, InputIterator last); // freestanding
    template<class InputIterator>
        constexpr InputIterator
            next(InputIterator x,
                typename iterator_traits<InputIterator>::difference_type n = 1); // freestanding
    template<class BidirectionalIterator>
        constexpr BidirectionalIterator
            prev(BidirectionalIterator x,
                typename iterator_traits<BidirectionalIterator>::difference_type n = 1); // freestanding

    // [predef.iterators], predefined iterators
    template<class Iterator> class reverse_iterator; // freestanding

    template<class Iterator1, class Iterator2>
        constexpr bool operator==(
            const reverse_iterator<Iterator1>& x,
            const reverse_iterator<Iterator2>& y); // freestanding
    template<class Iterator1, class Iterator2>
        constexpr bool operator!=(
            const reverse_iterator<Iterator1>& x,
            const reverse_iterator<Iterator2>& y); // freestanding
    template<class Iterator1, class Iterator2>

```

```

constexpr bool operator<(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y); // freestanding
template<class Iterator1, class Iterator2>
constexpr bool operator>(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y); // freestanding
template<class Iterator1, class Iterator2>
constexpr bool operator<=(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y); // freestanding
template<class Iterator1, class Iterator2>
constexpr bool operator>=(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y); // freestanding

template<class Iterator1, class Iterator2>
constexpr auto operator-(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y) -> decltype(y.base() - x.base()); // freestanding
template<class Iterator>
constexpr reverse_iterator<Iterator>
operator+(
    typename reverse_iterator<Iterator>::difference_type n,
    const reverse_iterator<Iterator>& x); // freestanding

template<class Iterator>
constexpr reverse_iterator<Iterator> make_reverse_iterator(Iterator i); // freestanding

template<class Container> class back_insert_iterator; // freestanding
template<class Container>
    back_insert_iterator<Container> back_inserter(Container& x); // freestanding

template<class Container> class front_insert_iterator; // freestanding
template<class Container>
    front_insert_iterator<Container> front_inserter(Container& x); // freestanding

template<class Container> class insert_iterator; // freestanding
template<class Container>
    insert_iterator<Container> inserter(Container& x, typename Container::iterator i); // freestanding

template<class Iterator> class move_iterator; // freestanding
template<class Iterator1, class Iterator2>
constexpr bool operator==(
    const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y); // freestanding
template<class Iterator1, class Iterator2>
constexpr bool operator!=(
    const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y); // freestanding
template<class Iterator1, class Iterator2>
constexpr bool operator<(
    const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y); // freestanding
template<class Iterator1, class Iterator2>
constexpr bool operator>(
    const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y); // freestanding
template<class Iterator1, class Iterator2>
constexpr bool operator<=(
    const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y); // freestanding
template<class Iterator1, class Iterator2>
constexpr bool operator>=(
    const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y); // freestanding

template<class Iterator1, class Iterator2>
constexpr auto operator-(
    const move_iterator<Iterator1>& x,
    const move_iterator<Iterator2>& y) -> decltype(x.base() - y.base()); // freestanding
template<class Iterator>
constexpr move_iterator<Iterator> operator+( // freestanding
    typename move_iterator<Iterator>::difference_type n, const move_iterator<Iterator>& x);
template<class Iterator>
constexpr move_iterator<Iterator> make_move_iterator(Iterator i); // freestanding

// [stream.iterators], stream iterators
template<class T, class charT, class traits = char_traits<charT>,
    class Distance = ptrdiff_t>
class istream_iterator;
template<class T, class charT, class traits, class Distance>

```

```

    bool operator==(const istream_iterator<T, charT, traits, Distance>& x,
        const istream_iterator<T, charT, traits, Distance>& y);
template<class T, class charT, class traits, class Distance>
    bool operator!=(const istream_iterator<T, charT, traits, Distance>& x,
        const istream_iterator<T, charT, traits, Distance>& y);

template<class T, class charT = char, class traits = char_traits<charT>>
    class ostream_iterator;

template<class charT, class traits = char_traits<charT>>
    class istreambuf_iterator;
template<class charT, class traits>
    bool operator==(const istreambuf_iterator<charT, traits>& a,
        const istreambuf_iterator<charT, traits>& b);
template<class charT, class traits>
    bool operator!=(const istreambuf_iterator<charT, traits>& a,
        const istreambuf_iterator<charT, traits>& b);

template<class charT, class traits = char_traits<charT>>
    class ostreambuf_iterator;

// [iterator.range], range access
template<class C> constexpr auto begin(C& c) -> decltype(c.begin()); // freestanding
template<class C> constexpr auto begin(const C& c) -> decltype(c.begin()); // freestanding
template<class C> constexpr auto end(C& c) -> decltype(c.end()); // freestanding
template<class C> constexpr auto end(const C& c) -> decltype(c.end()); // freestanding
template<class T, size_t N> constexpr T* begin(T (&array)[N]) noexcept; // freestanding
template<class T, size_t N> constexpr T* end(T (&array)[N]) noexcept; // freestanding
template<class C> constexpr auto cbegin(const C& c) noexcept(noexcept(std::begin(c)))
    -> decltype(std::begin(c)); // freestanding
template<class C> constexpr auto cend(const C& c) noexcept(noexcept(std::end(c)))
    -> decltype(std::end(c)); // freestanding
template<class C> constexpr auto rbegin(C& c) -> decltype(c.rbegin()); // freestanding
template<class C> constexpr auto rbegin(const C& c) -> decltype(c.rbegin()); // freestanding
template<class C> constexpr auto rend(C& c) -> decltype(c.rend()); // freestanding
template<class C> constexpr auto rend(const C& c) -> decltype(c.rend()); // freestanding
template<class T, size_t N> constexpr reverse_iterator<T*> rbegin(T (&array)[N]); // freestanding
template<class T, size_t N> constexpr reverse_iterator<T*> rend(T (&array)[N]); // freestanding
template<class E> constexpr reverse_iterator<const E*> rbegin(initializer_list<E> il); // freestanding
template<class E> constexpr reverse_iterator<const E*> rend(initializer_list<E> il); // freestanding
template<class C> constexpr auto crbegin(const C& c) -> decltype(std::rbegin(c)); // freestanding
template<class C> constexpr auto crend(const C& c) -> decltype(std::rend(c)); // freestanding

// [iterator.container], container access
template<class C> constexpr auto size(const C& c) -> decltype(c.size()); // freestanding
template<class T, size_t N> constexpr size_t size(const T (&array)[N]) noexcept; // freestanding
template<class C> [[nodiscard]] constexpr auto empty(const C& c) -> decltype(c.empty()); // freestanding
template<class T, size_t N> [[nodiscard]] constexpr bool empty(const T (&array)[N]) noexcept; // freestanding
template<class E> [[nodiscard]] constexpr bool empty(initializer_list<E> il) noexcept; // freestanding
template<class C> constexpr auto data(C& c) -> decltype(c.data()); // freestanding
template<class C> constexpr auto data(const C& c) -> decltype(c.data()); // freestanding
template<class T, size_t N> constexpr T* data(T (&array)[N]) noexcept; // freestanding
template<class E> constexpr const E* data(initializer_list<E> il) noexcept; // freestanding
}

```

Change in [algorithm.syn]:

Instructions to the editor:

Please add a **// freestanding** comment to all of the non-ExecutionPolicy overloads of the following functions:

- all_of
- any_of
- none_of
- for_each
- for_each_n
- find
- find_if
- find_if_not
- find_end
- find_first_of
- adjacent_find
- count
- count_if

- mismatch
- equal
- is_permutation
- search
- search_n
- copy
- copy_n
- copy_if
- copy_backward
- move
- move_backward
- swap_ranges
- iter_swap
- transform
- replace
- replace_if
- replace_copy
- replace_copy_if
- fill
- fill_n
- generate
- generate_n
- remove
- remove_if
- remove_copy
- remove_copy_if
- unique
- unique_copy
- reverse
- reverse_copy
- rotate
- rotate_copy
- sample
- shuffle
- shift_left
- shift_right
- sort
- partial_sort
- partial_sort_copy
- is_sorted
- is_sorted_until
- nth_element
- lower_bound
- upper_bound
- equal_range
- binary_search
- is_partitioned
- partition
- partition_copy
- partition_point
- merge
- includes
- set_union
- set_intersection
- set_difference
- set_symmetric_difference
- push_heap
- pop_heap
- make_heap
- sort_heap
- is_heap
- is_heap_until
- min
- max
- minmax
- min_element
- max_element
- minmax_element
- clamp
- lexicographical_compare
- compare_3way
- lexicographical_compare_3way

- `next_permutation`
- `prev_permutation`

Note that `stable_sort`, `stable_partition`, and `inplace_merge` are purposefully omitted.

Change in [numeric.ops.overview]:

Instructions to the editor:

Please add a `// freestanding` comment to all of the non-ExecutionPolicy overloads of the following functions:

- `accumulate`
- `reduce`
- `inner_product`
- `transform_reduce`
- `partial_sum`
- `exclusive_scan`
- `inclusive_scan`
- `transform_exclusive_scan`
- `transform_inclusive_scan`
- `adjacent_difference`
- `iota`
- `gcd`
- `lcm`

Change in [rand.synopsis]:

```

?..?                Header <random> synopsis                [rand.synopsis]

#include <initializer_list>

namespace std {
    // [rand.req.urng], uniform random bit generator requirements
    template<class G>
        concept UniformRandomBitGenerator = see below; // freestanding

    // [rand.eng.lcong], class template linear_congruential_engine
    template<class UIntType, UIntType a, UIntType c, UIntType m>
        class linear_congruential_engine; // freestanding

    // [rand.eng.mers], class template mersenne_twister_engine
    template<class UIntType, size_t w, size_t n, size_t m, size_t r,
              UIntType a, size_t u, UIntType d, size_t s,
              UIntType b, size_t t,
              UIntType c, size_t l, UIntType f>
        class mersenne_twister_engine; // freestanding

    // [rand.eng.sub], class template subtract_with_carry_engine
    template<class UIntType, size_t w, size_t s, size_t r>
        class subtract_with_carry_engine; // freestanding

    // [rand.adapt.disc], class template discard_block_engine
    template<class Engine, size_t p, size_t r>
        class discard_block_engine; // freestanding

    // [rand.adapt.ibits], class template independent_bits_engine
    template<class Engine, size_t w, class UIntType>
        class independent_bits_engine; // freestanding

    // [rand.adapt.shuf], class template shuffle_order_engine
    template<class Engine, size_t k>
        class shuffle_order_engine; // freestanding

    // [rand.predef], engines and engine adaptors with predefined parameters
    using minstd_rand0 = see below; // freestanding
    using minstd_rand = see below; // freestanding
    using mt19937 = see below; // freestanding
    using mt19937_64 = see below; // freestanding
    using ranlux24_base = see below; // freestanding
    using ranlux48_base = see below; // freestanding
    using ranlux24 = see below; // freestanding
    using ranlux48 = see below; // freestanding
    using knuth_b = see below; // freestanding

```

```

using default_random_engine = see below; // freestanding

// [rand.device], class random_device
class random_device;

// [rand.util.seedseq], class seed_seq
class seed_seq;

// [rand.util.canonical], function template generate_canonical
template<class RealType, size_t bits, class URBG>
    RealType generate_canonical(URBG& g);

// [rand.dist.uni.int], class template uniform_int_distribution
template<class IntType = int>
    class uniform_int_distribution; // freestanding

// [rand.dist.uni.real], class template uniform_real_distribution
template<class RealType = double>
    class uniform_real_distribution;

```

Change in [cmath.syn]:

```

// [c.math.abs], absolute values
int abs(int j); // freestanding
long int abs(long int j); // freestanding
long long int abs(long long int j); // freestanding
float abs(float j);
double abs(double j);
long double abs(long double j);

```

Change in [cinttypes.syn]:

?..?	Header <cinttypes> synopsis	[cinttypes.syn]
------	--	------------------------

```

#include <cstdint> // see [cstdint.syn]

namespace std {
    using imaxdiv_t = see below; // freestanding

    intmax_t imaxabs(intmax_t j); // freestanding
    imaxdiv_t imaxdiv(intmax_t numer, intmax_t denom); // freestanding
    intmax_t strtoumax(const char* nptr, char** endptr, int base);
    uintmax_t strtoumax(const char* nptr, char** endptr, int base);
    intmax_t wcstoumax(const wchar_t* nptr, wchar_t** endptr, int base);
    uintmax_t wcstoumax(const wchar_t* nptr, wchar_t** endptr, int base);

    intmax_t abs(intmax_t); // optional, see below, freestanding
    imaxdiv_t div(intmax_t, intmax_t); // optional, see below, freestanding
}

```

Acknowledgements

Thanks to Brandon Streiff, Joshua Cannon, Phil Hindman, and Irwan Djajadi for reviewing this proposal.

Thanks to Odin Holmes for helping publicize this paper, presenting it in Rapperswil, and providing feedback.

Similar work was done in the C++11 timeframe by Lawrence Cowl and Alberto Ganesh Barbati in [N3256](#).

CppCon talks on getting C++ support in various unusual environments:

[Baker2017] CppCon 2017: Billy Baker "Almost Unlimited Modern C++ in Kernel-Mode Applications"

[Quinn2016] CppCon 2016: Rian Quinn "Making C++ and the STL Work in the Linux / Windows Kernels"

[Bratterud2017] CppCon 2017: Alfred Bratterud "Deconstructing the OS: The devil's In the side effects"

[Meredith11] Conservative use of noexcept in the Library. A. Meredith; J. Lakos. [N3279](#).